UNIVERSIDADE FEDERAL FLUMINENSE

**JOÃO VICTOR DE SOUZA PIMENTEL CUNHA**

# COMO O CHATGPT PODE AUXILIAR A EDUCAÇÃO? Proposta de um framework de um chatbot no auxilio à sala de aula invertida

Rio das Ostras, RJ

2024

JOÃO VICTOR DE SOUZA PIMENTEL CUNHA

# COMO O CHATGPT PODE AUXILIAR A EDUCAÇÃO? Proposta de um framework de um chatbot no auxilio à sala de aula invertida

Monografia apresentada ao Curso de Bacharelado em Ciências da Computação do Instituto de Ciência e Tecnologia da Universidade Federal Fluminense, como requisito parcial para obtenção do Grau de Bacharel.

Orientador:
Prof.ª Leila Weitzel

Rio das Ostras, RJ

2024

Ficha catalográfica automática - SDC/BRO
Gerada com informações fornecidas pelo autor

S719c Souza Pimentel Cunha, João Victor de
COMO O CHATGPT PODE AUXILIAR A EDUCAÇÃO? : Proposta de um framework de um chatbot no auxilio à sala de aula invertida / João Victor de Souza Pimentel Cunha. - 2024.
57 f.: il.

Orientador: Leila Weitzel Coelho Da Silva.
Trabalho de Conclusão de Curso (graduação)-Universidade Federal Fluminense, Instituto de Ciência e Tecnologia, Rio das Ostras, 2024.

1. Inteligência Artificial. 2. Processamento de Linguagem Natural. 3. Grande Modelos de Linguagem. 4. Produção intelectual. I. Coelho Da Silva, Leila Weitzel, orientadora. II. Universidade Federal Fluminense. Instituto de Ciência e Tecnologia. III. Título.

CDD - XXX

Bibliotecário responsável: Debora do Nascimento - CRB7/6368

JOÃO VICTOR DE SOUZA PIMENTEL CUNHA

COMO O CHATGPT PODE AUXILIAR A EDUCAÇÃO? Proposta de um framework de um chatbot no auxilio à sala de aula invertida

Monografia apresentada ao Curso de Bacharelado em Ciências da Computação do Instituto de Ciência e Tecnologia da Universidade Federal Fluminense, como requisito parcial para obtenção do Grau de Bacharel.

Aprovada em Agosto de 2024.

BANCA EXAMINADORA

---

Prof. Leila Weitzel Coelho da Silva - Orientador, UFF

---

Prof. Dalessandro Soares Vianna, UFF

---

Prof. Roberto Gonçalves Pacheco, UFF

Rio das Ostras, RJ

2024

# Resumo

Esta pesquisa investiga a implementação de *chatbots*, especificamente o *ChatGPT* da OpenAI, em ambientes educacionais, com foco em aprimorar o aprendizado personalizado através do modelo de sala de aula invertida. Reconhecendo tanto o potencial quanto as limitações do *ChatGPT*, o estudo propõe o desenvolvimento de um framework com objetivo de abordar tópicos especializados e de nicho, neste caso programação em Phyton. A pesquisa explora aspectos teóricos como o modelo de sala de aula invertida, características e limitações do *ChatGPT*, o papel do Processamento de Linguagem Natural na Inteligência Artificial, a relação entre Processamento de linguagem natural e desenvolvimento de *Chatbots*, Aprendizado de Máquina, Aprendizado Profundo, Redes Neurais Artificiais e Modelos de *Large Language Model*. Uma metodologia de pesquisa criteriosa é empregada, apresentando um design experimental detalhado que abrange coleta e processamento de dados, extração e seleção de recursos. Esses passos são planejados para garantir a integridade e validade dos achados da pesquisa. Discussões relevantes são apresentadas, e direções para trabalhos futuros são sugeridas para aprimorar ainda mais a utilidade da estrutura desenvolvida. Este estudo oferece uma contribuição significativa para o crescente campo de pesquisa sobre *chatbots* na educação, apresentando um framework para a implementação otimizada de paradigmas da Inteligencia Artificial no ensino e aprendizado.

**Palavras-chave**: Aprendizado de máquina, *ChatGPT*, Transformers, Large Language Model.

# Abstract

This research investigates the implementation of chatbots, specifically OpenAI's ChatGPT, in educational environments. The focus is on enhancing personalized learning through the flipped classroom model. Recognizing both the potential and limitations of ChatGPT, the study proposes the development of a framework aimed at addressing specialized and niche topics, specifically Python programming. The research explores theoretical aspects such as the flipped classroom model, characteristics and limitations of ChatGPT, the role of Natural Language Processing in Artificial Intelligence, the relationship between Natural Language Processing and chatbot development, Machine Learning, Deep Learning, Artificial Neural Networks, and Large Language Models. A rigorous research methodology is employed, featuring a detailed experimental design encompassing data collection and processing, resource extraction, and selection. These steps are planned to ensure the integrity and validity of the research findings. Relevant discussions are presented, and directions for future work are suggested to further enhance the utility of the developed framework. This study offers a significant contribution to the growing field of research on chatbots in education, presenting a framework for the optimized implementation of Artificial Intelligence paradigms in teaching and learning.

**Keywords**: Machine Learning, chatGPT, Transformers, Large Language Model.

# Lista de Figuras


1 Esboço do projeto. . . . . 27

2 Fluxograma do treinamento. . . . . 30

3 Configuração de *fine-tuning*. . . . . 31

4 Respostas dos modelos para um mesmo *prompt*. . . . . 37

5 inicialização do *server*. . . . . 38

6 Exemplo de definiçao de rota. . . . . 39

7 Configuração do *container Docker*. . . . . 41

8 Interface gráfica. . . . . 42

9 Executar *Containers Docker* . . . . . 43

10 Documentação da API . . . . . 44

11 Tela do Chat . . . . . 45

12 Estados da barra lateral . . . . . 46

13 Área do Chat . . . . . 47

14 Resposta do chat . . . . . 48

# Lista de Tabelas

1 Similaridade com a base de dados MBPP. . . . . 34

2 Similaridade entre o GPT-3.5 e o Modelo do projeto. . . . . 35

# Sumário

**1 Introdução** **10**

1.1 Objetivo . . . . . 12

1.2 Motivação . . . . . 12

1.3 Organização da Monografia . . . . . 13

**2 Referencial Teórico** **14**

2.1 Modelo de aprendizagem – sala de aula invertida . . . . . 14

2.2 Características do *ChatGPT* . . . . . 15

2.3 Aprendizado de Máquina e Aprendizado Profundo . . . . . 15

2.3.1 Aprendizado de Máquina - *Machine Learning* . . . . . 16

2.3.2 Aprendizado Profundo - *Deep Learning* . . . . . 17

2.4 Processamento de Linguagem Natural - PLN . . . . . 19

2.5 A Evolução dos Modelos de Linguagem . . . . . 20

2.5.1 TF-IDF e N-grams . . . . . 20

2.5.2 O Surgimento dos *Embeddings* . . . . . 21

2.5.3 BERT . . . . . 21

2.5.4 *Large Language Models* . . . . . 21

**3 Trabalhos Relacionados** **23**

3.1 Tadrisi (2023) . . . . . 23

3.1.1 Principais Contribuições . . . . . 23

3.2 Zhang, Cheung e Cheung (2021) . . . . . 24

**4 Desenvolvimento do *Framework*** **26**

4.1 Etapas da Metodologia de desenvolvimento . . . . . 26

4.1.1 Primeira Etapa - Definir e Especializar Modelo de IA . . . . . 29

4.1.1.1 **Técnicas de Treinamento** . . . . . 31

4.1.1.2 **Avaliação do modelo com *Rouge-1*:** . . . . . 33

4.1.2 Segunda Etapa - Desenvolver API . . . . . 37

4.1.3 Terceira Etapa - Desenvolver Interface de Usuário . . . . . 41

**5 Exemplo de Uso** **43**

**6 Considerações Finais** **49**

6.1 Trabalhos Futuros . . . . . 50

**REFERÊNCIAS** **51**

# 1 Introdução

Este capítulo introduz o tema da pesquisa, apresentando o objetivo, motivação e ao final como está organizada a monografia.

Um *chatbot*, ou sistemas de diálogo, é uma ferramenta de software que interage com os usuários em um determinado tópico ou em um domínio específico de maneira natural e conversacional usando texto e voz. Para muitas finalidades diferentes, os *chatbots* têm sido usados em uma ampla variedade de domínios, incluindo marketing, atendimento ao cliente, suporte técnico, bem como educação e treinamento. Os desenvolvimentos atuais nesta área sugerem que a interação com tecnologias, seja por linguagem natural ou por fala, é possível porque a tecnologia se desenvolve e os usuários se tornam mais acostumados a interagir com entidades digitais. Em vez de criar um aplicativo de máquina inteligente semelhante ao humano, trata-se de criar assistentes digitais eficazes capazes de fornecer informações, responder a perguntas, discutir um tópico específico ou executar uma tarefa. (Clarizia *et al.*, 2018; Okonkwo; Ade-Ibijola, 2021).

Assistentes digitais pessoais como Siri da Apple, Alexa da Amazon, Cortana da Microsoft ou Assistant do Google estão na vanguarda da tecnologia de reconhecimento de voz e inteligência artificial. Esses assistentes utilizam técnicas de aprendizado de máquina e são capazes de gerenciar tarefas do dia a dia, como a priorização de e-mails, destaque de conteúdos e interações mais importantes, ajudando seus usuários a se tornarem mais eficientes. Embora esses sistemas respondam de maneira eficaz a solicitações dentro de um domínio específico, eles não são tão eficientes ao lidar com perguntas cujo padrão não corresponde às regras nas quais o *chatbot* foi treinado. (Al *et al.*, 2021).

Um avanço nas abordagens de *chatbots* é o *ChatGPT* (*Chat Generative Pre-trained Transformer*) da OpenAI[1], uma ferramenta projetada para automatizar várias tarefas, permitindo que os usuários tenham conversas semelhantes às humanas. O sistema é capaz de responder a perguntas e auxiliar em diversas atividades, além das mencionadas

[1]https://openai.com/blog/chatgpt

anteriormente, como escrever redações sobre determinados temas, codificar, reconhecer erros, questionar suposições incorretas e recusar solicitações inadequadas (especialmente em relação a temas sensíveis). O *ChatGPT* foi desenvolvido para fornecer respostas detalhadas a qualquer consulta digitada — desde perguntas até declarações. Embora os melhores resultados sejam obtidos ao se inserir uma declaração, perguntas diretas também podem ser feitas. Por exemplo, "explique como funciona um motor de combustão" obterá um resultado mais detalhado do que perguntar "como funciona um motor de combustão". Também é possível ser mais específico e solicitar um número exato de parágrafos para um ensaio. (Kemper, 2023).

O *ChatGPT* tem sido assunto recorrente especialmente entre os acadêmicos preocupados com o impacto na educação. Esse modelo será bom para o sistema educacional? Ou seria o ataque ao pensamento, bem como às habilidades de tomada de decisão de nossos alunos? Uma preocupação proeminente é que os alunos possam desenvolver inércia para aprender. Os acadêmicos temem que esta invenção possa ser o começo do fim das habilidades de pensamento humano. Entretanto, se for bem utilizada poderá fornecer aulas personalizadas e em tempo real, o que pode fazê-los entender melhor conceitos complexos e melhorar suas notas (Roose, 2023). Todavia, se os alunos utilizarem esta ferramenta sem uma orientação para obter respostas e soluções, eles podem ficar menos motivados a aprender de forma independente. Neste sentido as instituições acadêmicas terão que desenvolver métodos compatíveis para avaliar as lacunas de aprendizagem dos alunos e potenciais futuros. É essencial estabelecer a aplicação sensata da tecnologia por todas as partes interessadas, especialmente no domínio da educação, para garantir que ela melhore e apoie o crescimento, a aprendizagem e o desenvolvimento humanos. (Clarizia *et al.*, 2018; Kuhail *et al.*, 2023; Okonkwo; Ade-Ibijola, 2021; Wollny *et al.*, 2021).

O *ChatGPT* está sendo cada vez mais usados para melhorar a interação dos alunos neste mundo atual de tecnologia, onde a comunicação e muitas outras atividades dependem fortemente da plataforma online. O *ChatGPT* pode fornecer instantaneamente aos alunos detalhes padronizados, como conteúdo do curso, perguntas e respostas, materiais de estudo, aprendendo a entender os conceitos das linguagens de programação entre outros. Esses sistemas podem não apenas melhorar o envolvimento e o suporte dos alunos dando maior autonomia, mas também fornecer aos alunos um ambiente de aprendizado mais personalizado, além de ser um mecanismo de diálogo que estimula a aprendizagem colaborativa. Embora o*ChatGPT* possua muitos pontos fortes, sendo um modelo generalizado, nem sempre é a solução mais eficaz para tópicos mais restritos e especializados (Kuhail *et al.*, 2023; Okonkwo; Ade-Ibijola, 2021). Apesar da empresa OpenAI observar

que dificilmente ocorram inconsistências nas respostas obtidas, a empresa também afirma que a ferramenta possui atualmente "conhecimento limitado aos eventos mundiais após 2021", justamente porque foi treinada com dados antes desta data (Ribeiro, 2023; Sha, 2023). O *ChatGPT* foi treinado em diversos idiomas, mas a quantidade de dados em inglês é significativamente maior. Isso pode levar a resultados menos precisos em outras línguas. Outra limitação é o tamanho máximo de entrada, que é de 4.096 tokens. Um token corresponde aproximadamente a uma palavra, mas pode variar dependendo do contexto. Textos que excedem esse limite são truncados sem aviso prévio. Por fim, é importante ressaltar que o *ChatGPT* responde melhor a comandos claros e concisos, sem a necessidade de frases elaboradas.(Kemper, 2023).

## 1.1 Objetivo

Para mitigar esses problemas é que se propõe um *framework* que possa direcionar os alunos a uma resposta plausível às suas arguições auxiliando tanto professores quanto alunos na modalidade de sala de aula invertida. Sendo assim, a presente proposta visa criar um *framework* para um aplicativo educacional mais inteligente, personalizado e eficaz, aproveitando os avanços mais recentes da tecnologia da Inteligência Artificial tais como o*ChatGPT*. A ideia deste estudo é aliar o sistema de ensino com o uso do *ChatGPT* através da elaboração de uma ferramenta que sirva para orientar os alunos e otimizar o aprendizado, fazendo com que as instituições de ensino se adaptem e mudem o ponto de vista sobre o seu uso em sala de aula.

## 1.2 Motivação

A inteligência artificial, em especial os modelos de linguagem como o *ChatGPT*, estão revolucionando diversos setores, e a educação não é exceção. Ferramentas como o *ChatGPT* são capazes de gerar textos coerentes e informativos em resposta a uma ampla variedade de prompts, apresentando potencial para transformar a maneira como aprendemos e ensinamos. Uma das principais vantagens do *ChatGPT* na educação é a sua capacidade de fornecer respostas instantâneas e personalizadas a dúvidas dos alunos. Imagine um estudante com uma pergunta sobre um conceito complexo de física fora do horario da aula, à noite por exemplo. Com o *ChatGPT*, ele pode obter uma explicação clara e concisa a qualquer hora, sem precisar esperar até a próxima aula. Essa disponibilidade constante estimula a curiosidade e a autonomia dos aprendizes, permitindo que eles aprofundem

seus conhecimentos no próprio ritmo. (Roos, 2018).

Além disso, o *ChatGPT* pode ser utilizado para criar materiais didáticos personalizados. Ao analisar o histórico de aprendizado de um aluno, o modelo pode gerar exercícios, exemplos e explicações que se adaptam ao seu nível de conhecimento e estilo de aprendizado. Essa personalização torna o processo de ensino mais eficaz e engajador, pois cada aluno recebe o suporte que precisa no momento certo. Outra aplicação interessante do *ChatGPT* é na criação de simulações e jogos educativos. Ao interagir com o modelo, os alunos podem praticar habilidades como resolução de problemas, tomada de decisões e comunicação de forma lúdica e divertida. Essas experiências imersivas tornam o aprendizado mais atrativo e memorável. No entanto, é importante ressaltar que o *ChatGPT* não deve ser visto como um substituto para o professor. O papel do educador continua sendo fundamental para guiar o aprendizado, estimular o pensamento crítico e promover a colaboração entre os alunos. O *ChatGPT* é uma ferramenta que complementa o trabalho do professor, oferecendo novas possibilidades e recursos para a criação de experiências de aprendizado mais significativas. (Roose, 2023).

O *ChatGPT* representa uma ferramenta poderosa. Sua capacidade de fornecer informações personalizadas, criar materiais didáticos adaptados e promover o aprendizado ativo o torna uma ferramenta para professores e alunos. Ao utilizar essa tecnologia de forma estratégica, pode-se criar ambientes de aprendizado mais dinâmicos, engajadores e eficazes. (Roose, 2023).

## 1.3 Organização da Monografia

Além do capítulo 1 introdutório que discute o tema, objetivo da pesquisa e a motivação, os Capítulos seguintes estão organizados da seguinte forma: capítulo 2 aborda o referencial teórico da pesquisa, onde são tratados os conceitos. Esses paradigmas têm demonstrado resolver efetivamente várias tarefas de processamento de linguagem. O capítulo 3 discute os trabalhos que nortearam esta pesquisa, mostrando uma revisão dos principais trabalhos publicados na literatura. No capítulo 4 aborda-se a metodologia da pesquisa, como se deu a coleta, tratamento, a extração dos recursos, os modelos de DL utilizados. O capítulo 5 mostra os resultados verificados, além dos resultados da performance das redes profundas. Por fim, apresenta-se a conclusão e a possibilidade de trabalhos futuros no capítulo 6.

# 2 Referencial Teórico

Neste capítulo são abordados os principais conceitos utilizados nesta pesquisa. A primeira seção (2.1) explora o conceito de metodologias ativas e do modelo de aprendizagem de sala de aula invertida. Em seguida, a seção 2.2 aborda as características do *ChatGPT*, abordando suas capacidades. A seção 2.3 se concentra no aprendizado de máquina, aprendizado profundo e redes neurais. A seção 2.4 examina o Processamento de Linguagem Natural (PLN), incluindo o objetivo e sub-áreas, além de apresentar um breve histórico. Por fim, a seção 2.5 aborda a evolução dos modelos de linguagem.

## 2.1 Modelo de aprendizagem – sala de aula invertida

A aplicação de metodologias ativas no ensino pode ser desafiadora, pois exige uma mudança de postura tanto do aluno quanto do professor. Diferentemente do método tradicional, que se baseia na transferência de conhecimento, onde se espera que o aluno aprenda e o professor ensine, com o foco principal no professor, as metodologias ativas propõem uma mudança nesse padrão. Elas colocam os alunos como foco principal, envolvendo-os em atividades e debates, e estimulando o pensamento crítico e o trabalho em equipe. Pesquisas indicam que a aprendizagem ativa contribui significativamente para uma melhor compreensão e retenção de informações. Além disso, ela se mostra eficaz no desenvolvimento de habilidades cognitivas de nível superior, como a resolução de problemas e o pensamento crítico. (Konopka; Adaime; Mosele, 2015).

A sala de aula invertida é uma metodologia ativa em que há uma inversão no modelo de ensino, onde o que geralmente se faz em casa passa a ser feito em sala e vice-versa. A sala de aula invertida começou a ser desenvolvida nos anos 90 com trabalhos de pesquisadores das universidades americanas de Harvard e Yale. Em 2000, o pesquisador J.Wesley Baker (Baker, 2001) avançou nos estudos acerca desse modelo de educação inovador e apresentou o conceito de sala de aula invertida. Este modelo é bastante utilizado em ambiente acadêmico, nele conteúdos geralmente utilizados em sala, como textos e video-

aulas, são disponibilizados préviamente de forma online para ser estudado em casa. Após isso, durante o horario de aula, os professores criam estudos de caso baseados no que foi estudado pelos alunos (em casa) e estimulam sessões de perguntas e respostas, discussões e esclarecimento de dúvidas. É um modelo que depende de um bom planejamento, participação, *feedback* e avaliação dos conteúdos. (Bergmann; Sams, 2012; Sabec *et al.*, 2020).

## 2.2 Características do *ChatGPT*

Nos últimos anos, a inteligência artificial (IA) tem experimentado um crescimento acelerado, demonstrando-se como um tópico relevante para diversas áreas. A IA objetiva a capacidade de máquinas em realizar tarefas que tipicamente exigem inteligência humana, como o raciocínio, aprendizado e a resolução de problemas. Entre os avanços mais significativos recentemente, destacam-se os grandes modelos de linguagem (LLMs), como o *ChatGPT*. O *ChatGPT* foi desenvolvido e lançado pela OpenAI ao final de 2022, rapidamente se popularizando. Se trata de uma inteligência artificial generativa, que possui grandes capacidades na compreensão e geração de linguagem natural. Ou seja, o *ChatGPT* consegue realizar uma diversidade de tarefas, além de conseguir se envolver em conversas sobre variados tópicos, ele possui a capacidade de processar e gerar respostas em linguagem natural de forma interativa. É uma ferramenta versátil, sendo adaptativa, ou seja consegue se adaptar as necessidades do usuário, além de poder ser utilizado em diversos cenários e tarefas relacionadas ao processamento de linguagem natural (PLN), como tradução, resumo de textos e geração de conteúdo. Muitas dessas capacidades se devem ao pré-treinamento com um conjunto de dados grande e diverso o suficiente, fazendo com que essas LLMs, como o GPT-3, tenham boa performance em diferentes domínios.(Tadrisi, 2023; Ali *et al.*, 2024; Radford *et al.*, 2019).

## 2.3 Aprendizado de Máquina e Aprendizado Profundo

De acordo com (Chollet, 2018), o Aprendizado de Máquina (*Machine Learning* - ML) e o Aprendizado Profundo (*Deep Learning* - DL) representam abordagens do campo da IA, onde ambas possuem capacidade de atuar e solucionar problemas reais e mais complexos. Sendo o DL, de forma mais específica, um tipo de ML. Nas seções seguintes são discutidas as estratégias com maior profundidade.

### 2.3.1 Aprendizado de Máquina - *Machine Learning*

O ML é uma área que teve um grande crescimento nas últimas décadas, onde antes era focado em estudos e trabalhos acadêmicos, começou a ser utilizado no contexto comercial de maneira ampla, como por exemplo, para tarefas como visão computacional, processamento de linguagem natural, entre outros (Jordan; Mitchell, 2015).

Uma definição para ML é dada por Mitchell (1997), que afirma que é um programa de computador que aprende a partir da experiência **E**, referente a uma determinada categoria de tarefas **T**, utilizando uma medida de desempenho **P**, se seu desempenho, ao realizar tarefas **T** e mensurado por **P**, se aprimora com a experiência **E**. Deve-se entender a tarefa **T**, como a capacidade do modelo de realizar a tarefa especificada e não o aprendizado que o leva a realizá-la. O desempenho **P**, é uma medida quantitativa usada para avaliar o comportamento do modelo ao realizar a tarefa **T** proposta. Além disso, o desempenho **P** é avaliado a partir de dados nunca "vistos" pelo modelo, definido como conjunto teste. Já a experiência **E**, é definida pelo conjunto de dados e pelo tipo de treinamento escolhido.

Existem três tipos principais de aprendizado de máquina: aprendizado supervisionado, aprendizado não supervisionado e aprendizado por reforço. O **Aprendizado Supervisionado** ocorre com um conjunto de dados rotulados com valores de saída conhecidos, o modelo deve aprender a associar um tipo de entrada com um tipo de saída, assim consegue inferir respostas para os dados não conhecidos (dados de um conjunto de testes). Existe ainda a ideia de um supervisor, sendo o responsável pela rotulação dos dados. As principais tarefas que utilizam esse tipo de treinamento são as de Classificação e de Regressão. (Badillo *et al.*, 2020).

O **Aprendizado não Supervisionado**, diferentemente do método anterior, ocorre a partir de um conjunto de dados não rotulados, não necessitando de um supervisor. A ideia desse aprendizado é observar as particularidades dos dados de entrada, reconhecer padrões entre eles e agrupá-los (Alpaydin, 2020). Esse agrupamento pode servir, por exemplo, para agrupar diferentes tipos de peças de arte, agrupando por seus estilos de pintura.

No **aprendizado por reforço** ensina o modelo a tomar decisões visando os melhores resultados, imitando o aprendizado por tentativa e erro dos humanos. Ele utiliza um sistema de recompensa e punição para ajustar as ações do software, aprendendo com o feedback e identificando os melhores caminhos para atingir os objetivos, obtendo melhor pontuação (AWS, 2024a). Um exemplo de uso é a implementação de um algoritmo para

o jogo da cobra (Snake game), as boas políticas podem ser definidas como a quantidade de frutas ingeridas e a duração de tempo sem colidir com as paredes. Então com o tempo o modelo seria capaz de aprender com as sequências boas e melhorar seu desempenho no jogo.

Existem outras técnicas que se adequam a necessidade de uso, como o aprendizado **Auto-Supervisionado**, onde se utiliza do aprendizado não supervisionado para tarefas que geralmente precisam de aprendizado supervisionado. Nesta técnica os modelos geram os rótulos a partir dos dados não estruturados, deixando de depender de conjuntos de dados rotulados para a supervisão. Esta abordagem costuma ser mais econômica e consumir menos tempo, já que rotular o conjunto de dados demandam tempo e esforço humano. (Bergmann, 2023b).

### 2.3.2 Aprendizado Profundo - *Deep Learning*

O *Deep Learning*, apesar de parecer algo novo, suas ideias principais datam desde a década de 1940, onde há os primeiros indícios das redes neurais com McCulloch e Pitts (1943), baseando-se na biologia, mais especificamente nas atividades do sistema nervoso e suas redes de neurônios. Neste trabalho, essas redes são ditadas pelo caráter "tudo ou nada", ou seja, os neurônios podem assumir dois estados, positivo ou negativo (ativo ou inativo). Os estados desses neurônios dependem de um limite, que indica a excitação dos mesmos, determinando o seu estado. Posteriormente tem-se mais um avanço na área das redes neurais artificiais, com o trabalho de Rosenblatt (1958), no desenvolvimento do modelo Perceptron. O Perceptron segue o mesmo princípio do "tudo ou nada", sendo um modelo binário, porém neste modelo é abordado o conceito de pesos ajustáveis, onde os pesos dos neurônios eram ajustados através de conexões de feedback (conexões que seguem o fluxo contrário ao padrão, "voltando") ligadas dos neurônios de saída aos seus anteriores diretos. O ajuste de pesos é realizado de forma simples, se o modelo produzir uma resposta esperada, será enviado um feedback positivo (uma recompensa), caso contrário, um feedback negativo (uma punição), assim ajustando os pesos que levaram a tal resposta e melhorando suas previsões. Inicialmente, o Perceptron impactou de forma muito positiva no campo das redes neurais, porém por ser um modelo linear possui diversas limitações. Tais limitações levaram a uma decadência na área. (Goodfellow; Bengio; Courville, 2016).

Em meados dos anos 80, foi realizado um trabalho que acabou por resgatar o interesse na área: a implementação do algoritmo chamado *Backpropagation*, por Rumelhart, Hinton e Williams (1986), inspirado no estudo de Linnainmaa (1976). O algoritmo possui o

mesmo intuito do perceptron: ajustar os pesos para reduzir as diferenças entre o resultado esperado e o resultado do modelo. No entanto, o *Backpropagation* permite o treinamento de redes com mais camadas internas, pois possibilita o ajuste dos pesos dessas camadas internas, ou seja, que não pertencem à camada de entrada ou saída. Este algoritmo é uma abordagem utilizada até hoje para o treinamento de redes neurais profundas, mostrando, assim, sua importância.

Na década de 90 a popularidade das redes neurais volta a cair devido a problemas como The *vanishing gradient problem* (O problema do gradiente que desaparece), em modelos que utilizavam a abordagem *Backpropagation*, onde o erro, ao ser propagado para trás, vai se perdendo, ou seja, dificultando o aprendizado dos neurônios nas camadas internas mais iniciais e aumentando consideravelmente o tempo de treinamento (Hochreiter, 1998). Outros aspectos são as limitações de hardware da época e o avanço em outros campos do *Machine Learning*, não relacionados a Redes Neurais Artificiais.

Apenas nos anos 2000, houve uma retomada no crescimento da popularização das Redes Neurais, onde acabou por surgir o termo *Deep Learning* (Aprendizado Profundo). O *Deep learning* foi Impulsionado por alguns fatores (Goodfellow; Bengio; Courville, 2016):

- Pelo desenvolvimento do hardware, que proveu mais poder computacional para o treinamento das Redes Neurais Artificiais, como o uso de GPUs;
- Avanços em algoritmos e métodos que permitiram o treinamento de sucesso em Redes Neurais Profundas; e
- O aumento na quantidade de dados disponíveis.

O DL se tornou uma ferramenta amplamente utilizada, no momento em que permitiu que as redes neurais ficassem cada vez mais profundas, com mais camadas de diferentes arquiteturas e mais neurônios em suas camadas. Existem alguns pontos importantes que levaram à evolução do DL. Em 2006, o desenvolvimento do algoritmo *greedy layer-wise* (Guloso camada por camada), que treina uma camada interna por vez e é utilizado para o treinamento de *Deep Belief Networks* (Redes de Crença Profunda), otimiza o modelo ao iniciar os pesos próximos a um mínimo local devido ao agrupamento. Após o modelo pré-treinado, pode ser feito o *fine-tuning* (ajuste fino), onde um algoritmo busca a solução ideal para o problema especificado, como uma especialização. O método acabou por se provar uma abordagem mais eficiente, também, para o treinamento de outros tipos de

Redes Profundas. (Hinton; Osindero; Teh, 2006; Bengio *et al.*, 2006; LeCun; Bengio; Hinton, 2015).

O uso de Unidades de Processamento Gráfico GPUs também foi essencial para o DL, possibilitando que os pesquisadores treinaram o modelo de maneira muito mais rápida, já que são capazes de aplicar uma paralelização de forma mais eficiente, como demonstrado por Raina, Madhavan e Ng (2009). (LeCun; Bengio; Hinton, 2015).

O constante aumento na quantidade e disponibilidade de dados, o qual é importante para o ML permitiu o treinamento com grandes conjuntos de dados. Visto em trabalhos como "The Unreasonable Effectiveness of Data", por Halevy, Norvig e Pereira (2009), que trata a quantidade de dados como algo mais importante que a própria complexidade do modelo.

## 2.4 Processamento de Linguagem Natural - PLN

Linguagens naturais, que representam as formas de comunicação humanas, são linguagens ambíguas, não tão bem definidas e estão constantemente se modificando, geração após geração. Além disso, as linguagens humanas não são apenas conjuntos de caracteres, fazem parte delas conceitos, ideias, representações, que são coisas muito abstratas. Essas características das linguagens naturais, acabam por determinar as dificuldades enfrentadas pelo Processamento de Linguagem Natural (PLN). O PLN é uma área da Inteligência Artificial que objetiva, por parte da máquina, compreender e reproduzir a linguagem humana. O foco da área de PLN é o desenvolvimento de estratégias e algoritmos capazes de realizar tais conquistas. (Goldberg, 2022).

O PLN pode ser dividido em 2 subconjuntos, sendo a compreensão da linguagem natural (CLN) e a geração de linguagem natural (GLN). A CLN é responsável por tentar fazer com que as máquinas entendam a linguagem natural. Para isso, utiliza de elementos da linguística, como fonologia, morfologia, sintaxe, semantica, entre outros. Com isso permite que as máquinas possam analisar, extrair conceitos, emoções e palavras-chave. Já a GLN é responsável por fazer com que as máquinas tenham capacidade de produzir linguagem natural, como frases, sentenças e paragráfos. Utiliza de conceitos como organização textual, seleção de conteúdo, recursos linguísticos e realização. (Khurana *et al.*, 2023).

O PLN teve início na década de 1940 com trabalhos de tradução automática. Após um período de inatividade na pesquisa devido ao relatório ALPAC de 1966, alguns sistemas de

tradução automática começaram a fornecer resultados. Nos anos 1960, surgiram sistemas de perguntas e respostas como LUNAR, avançando nas capacidades de processamento de tarefas e de linguística. Nos anos 1980, a pesquisa em teoria gramatical computacional e o desenvolvimento de ferramentas como o *SRI's Core Language Engine*, uma ferramenta de proposito geral para processamento de frases, marcaram avanços importantes. A partir dos anos 1990, o foco se voltou para o processamento estatístico da linguagem, com a extração de informações e resumo automático sendo os tópicos principais. Já no início dos anos 2000, houveram avanços como a modelagem neural de linguagem e a introdução de redes neurais nas tarefas de PLN, como as redes neurais recorrentes (RNN) e as redes neurais convolucionais (CNN). além da criação de uma alternativa para representação do texto, chamada *embeddings*. Por fim os mecanismos de atenção e o uso de redes *transformers*, revolucionaram a área, permitindo a modelagem de dependências de longo prazo. (Khurana *et al.*, 2023).

## 2.5 A Evolução dos Modelos de Linguagem

A capacidade de processar e entender linguagem natural é um objetivo crucial para a inteligência artificial. Ao longo dos anos, os modelos de linguagem evoluíram significativamente, passando de métodos simples de contagem de palavras a sistemas complexos que podem gerar textos semelhantes ao humano. Esta jornada nos leva desde os primórdios do TF-IDF e n-gramas até os poderosos LLMs (*Large Language Models*) de hoje.

### 2.5.1 TF-IDF e N-grams

Os primeiros modelos de linguagem, como o TF-IDF (*Term Frequency-Inverse Document Frequency*) e n-gramas, eram baseados em estatísticas simples. O TF-IDF calculava a importância de uma palavra em um documento, considerando sua frequência e sua raridade em outros documentos. Os n-gramas, por outro lado, analisavam sequências de palavras adjacentes para identificar padrões e probabilidades de ocorrência. Esses métodos, apesar de simples, foram essenciais para tarefas como recuperação de informações e análise de sentimentos. (Seno *et al.*, 2023).

### 2.5.2 O Surgimento dos *Embeddings*

Com o avanço da computação e a popularização do aprendizado de máquina, os embeddings surgiram como uma nova forma de representar palavras e frases. Eles mapeiam palavras para vetores numéricos em um espaço multidimensional, capturando relações semânticas entre elas. Word2Vec e GloVe são exemplos notáveis de técnicas de embeddings, que permitiram aos modelos de linguagem entender o significado contextual das palavras e realizar tarefas como tradução automática e análise de similaridade. (Seno *et al.*, 2023).

### 2.5.3 BERT

A arquitetura *transformer*, introduzida em 2017, revolucionou o campo dos modelos de linguagem. Ao invés de depender de uma ordem sequencial de processamento, os *transformers* usam mecanismos de atenção para capturar relações complexas entre palavras em uma frase. BERT (*Bidirectional Encoder Representations from Transformers*) foi um dos primeiros modelos de linguagem baseados em *transformers*, e alcançou resultados impressionantes em diversas tarefas, como classificação de textos e resposta a perguntas. Outros modelos, como GPT-3, GPT-4 e LaMDA, seguiram os passos de BERT, utilizando recursos computacionais cada vez maiores para alcançar performances ainda mais avançadas. (Paes; Vianna; Rodrigues, 2023).

### 2.5.4 *Large Language Models*

Os LLMs (*Large Language Models*) são modelos de linguagem com bilhões de parâmetros, treinados em conjuntos de dados muito grandes. Eles demonstram uma capacidade impressionante de gerar textos criativos, traduzir idiomas, responder a perguntas complexas e até mesmo escrever códigos. Modelos como *ChatGPT* e Bard, baseados em LLMs, estão transformando a forma como interagimos com a tecnologia, oferecendo interfaces conversacionais intuitivas e ferramentas poderosas para diversas aplicações. Os LLM são reconhecidos por possuirem alto desempenho em uma variedade de tarefas do PLN. Elas utilizam redes neurais profundas, comunmente baseada na arquitetura *transformers*, com uma quantidade de parâmetros na faixa dos bilhões. Cada um desses parâmetros pode ser ajustado durante o treinamento e aprimorados pelos mecanismos de atenção. Novamente, esses modelos são pré-treinados com enormes quantidades de dados textuais não rotulados utilizando um tipo de aprendizado de máquina chamado aprendizado autossupervisionado (*self-supervised*). Um aspecto importante desses modelos é que, após o pré-treinamento,

é possível seguir algumas direções, como o *fine-tuning* e o *instruction tuning*. (Fan *et al.*, 2023; IBM, 2024)

Tanto o *fine-tuning* quanto o *instruction tuning* são técnicas que permitem aproveitar o conhecimento de um modelo pré-treinado para realizar novas tarefas, sendo ambas técnicas de transferência de aprendizado. O *fine-tuning* é um treinamento adicional, em modelos pré-treinados, com conjuntos de dados relacionados a uma tarefa específica, a partir de ajustes nos pesos do modelo. Essa técnica pode ser utilizada para ajustar os pesos de toda a rede neural; entretanto, nem sempre é a melhor opção por ser mais custosa. A utilidade do *fine-tuning* se dá pela facilidade e pelo baixo custo para especializar um modelo base, já que aproveita o pré-treinamento do modelo, consequentemente reduzindo o custo computacional e a quantidade de dados necessária (Bergmann, 2024). Já o *instruction tuning* é uma forma de *fine-tuning*, que possui valor devido às LLMs pré-treinadas que não terem boa otimização para conversação ou para seguir instruções. Então, esse método se trata de um *fine-tuning* com um conjunto de dados rotulados com instruções, ensinando o modelo a realizar tarefas específicas e a seguir comandos (Bergmann, 2023a).

# 3 Trabalhos Relacionados

Nesta seção, são discutidos os trabalhos que fundamentaram esta pesquisa.

De acordo com Wartman e Combs (2018), a educação está se transformando em resposta às mudanças no setor profissional, o que demanda o uso de Inteligência Artificial (IA) no ensino e na aprendizagem. No contexto de tutoria, um chatbot pode ser utilizado para modelar o estilo de aprendizagem de um aluno, empregando linguagem natural para prever e personalizar suas sessões de aprendizado (Crockett; Latham; Whitton, 2017). Estudos adicionais (Lin; Chang, 2020; Murad *et al.*, 2019) destacam que chatbots são inovações tecnológicas valiosas que aumentam o interesse dos alunos em aprender, além de melhorar a aquisição de habilidades cognitivas e o desempenho acadêmico.

Com este entendimento, dois estudos se destacaram e influenciaram este trabalho.

## 3.1 Tadrisi (2023)

No trabalho de Tadrisi (2023), há o desenvolvimento de uma aplicação de IA com o objetivo de aprender com postagens de blog e fornecer respostas relevantes para as consultas dos usuários. O autor explora o modelo de linguagem da *OpenAI*, o *GPT-3*, utilizando *Python* para desenvolver uma aplicação interativa de aprendizado, utilizando dados externos para melhorar a precisão do modelo e obter respostas com informações atualizadas.

### 3.1.1 Principais Contribuições

O autor propõe uma abordagem de três etapas para o desenvolvimento da aplicação, sendo elas:

- o treinamento do modelo de IA com uma postagem específica, utilizando o *GPT-3* e a ferramenta *LlamaIndex* para conectar do dados externos ao modelo;

- criação de uma API para fornecer respostas com base no conhecimento do modelo. Desenvolvida com o *framework Django* para servir a API e implementar as rotas; e
- construção de uma interface onde os usuários possam fazer suas perguntas e ver as respostas, onde utiliza o pacote *http.server*, do *Python*, para servir o HTML já estilizado com *TailwindCSS*.

A abordagem proposta por Tadrisi (2023), para o desenvolvimento da aplicação, foi adotada neste trabalho, com diferenças em cada fase. As principais diferenças incluem o uso de ferramentas distintas para a construção da API e para a criação da interface do usuário, a utilização de técnicas de containerização, e, principalmente, a escolha e o treinamento do modelo de IA. Este trabalho priorizou modelos open-source, que não geram custos de uso, para garantir acessibilidade e flexibilidade na implementação, além de utilizar o *fine-tuning* para especializar modelos pré-treinados.

## 3.2 Zhang, Cheung e Cheung (2021)

Este trabalho observa o aumento da popularidade do modelo de sala de aula invertida, notando a quantidade de estudos práticos que objetivam entender a relação entre o modelo de sala de aula invertida e o desempenho acadêmico dos alunos. Os autores, para investigar mais profundamente o impacto da sala de aula invertida no desempenho acadêmico de estudantes universitários, adotaram o método de meta-análise para analisar 20 estudos experimentais, nacionais e internacionais, sobre a sala de aula invertida.

Neste trabalho foram investigados estudos variados. Alguns concluiram que a sala de aula invertida causava impacto positivo no aprendizado dos alunos, sendo superior ao modelo tradicional. Outros estudos concluindo que salas de aula tradicionais são superiores às salas de aula invertidas. Por fim, estudos concluindo que ambos os modelos de aula tiveram o mesmo efeito na performance dos estudantes.

Os autores optaram por adotar o método de meta-análise que acaba por aprimorar a pesquisa tradicional a partir de métodos específicos de design e estatística. Esse método de pesquisa reúne resultados de pesquisas relacionados e independentes e utiliza métodos estatísticos para analisá-los. Com isso utilizaram os seguintes critérios para seleção para os estudos:

- "os estudos devem examinar o impacto da sala de aula invertida no desempenho

dos alunos, utilizando o ensino invertido como a principal variável independente de pesquisa";

- "os estudos devem ser pesquisas experimentais ou quase-experimentais com dados de pré-teste e pós-teste. Estudos com uma diferença de pré-teste superior a 50% do desvio padrão são excluídos";
- "os estudos devem incluir estatísticas suficientes, como tamanhos de amostra, médias e desvio padrão, para calcular o tamanho do efeito"; e
- "os participantes dos estudos devem ser estudantes universitários de instituições de ensino superior. O tamanho da amostra deve ser superior a 25 pessoas, e os tamanhos das amostras dos dois grupos devem ser semelhantes para garantir a precisão do experimento".

Além disso, a diferença média padronizada é empregada como *effect size*, com o intuito de padronizar os resultados dos estudos para uma comparação direta. Após a realização da análise, os autores concluem que existe uma correlação positiva entre o desempenho dos estudantes e o modelo de sala de aula invertida, sendo considerado um efeito positivo para o desenvolvimento dos mesmos, independentemente do tamanho da turma, embora tenha maior efeito em turmas de tamanho médio. Eles inferem que o modelo pode estimular e aumentar o interesse dos estudantes em aprender. Alertam que este modelo de ensino pode ser influenciado por diversos fatores, como a quantidade de alunos, o tipo de matéria e o tipo de conhecimento. Determinam que o impacto da sala de aula invertida é maior em turmas de tamanho médio, em estudos na área de ciências e em aplicações práticas. Por fim, expressam a necessidade de estimular a iniciativa dos próprios alunos para a independência no aprendizado.

Levando em consideração os resultados obtidos, este estudo foi um dos motivadores para a utilização da metodologia de sala de aula invertida neste trabalho.

# 4 Desenvolvimento do *Framework*

Neste capítulo, são apresentadas as etapas do projeto realizadas para atingir o objetivo proposto. Inicialmente, são apresentadas as fases de pré-desenvolvimento, seguidas das etapas que foram realizadas, que abrangem a especialização de um modelo de IA, o desenvolvimento de uma API para utilizar o modelo e a criação de uma interface para interação direta com o usuário.

## 4.1 Etapas da Metodologia de desenvolvimento

A Figura 1 mostra um esboço inicial idealizado para este trabalho, demonstrando os passos necessários antes, durante e após o desenvolvimento de cada etapa. É possível notar que este trabalho se divide em três etapas principais, onde os blocos em roxo representam as etapas de pré-desenvolvimento presentes nas três fases, os blocos em azul correspondem aos passos de desenvolvimento, e os blocos em cinza indicam as entregas de cada uma.

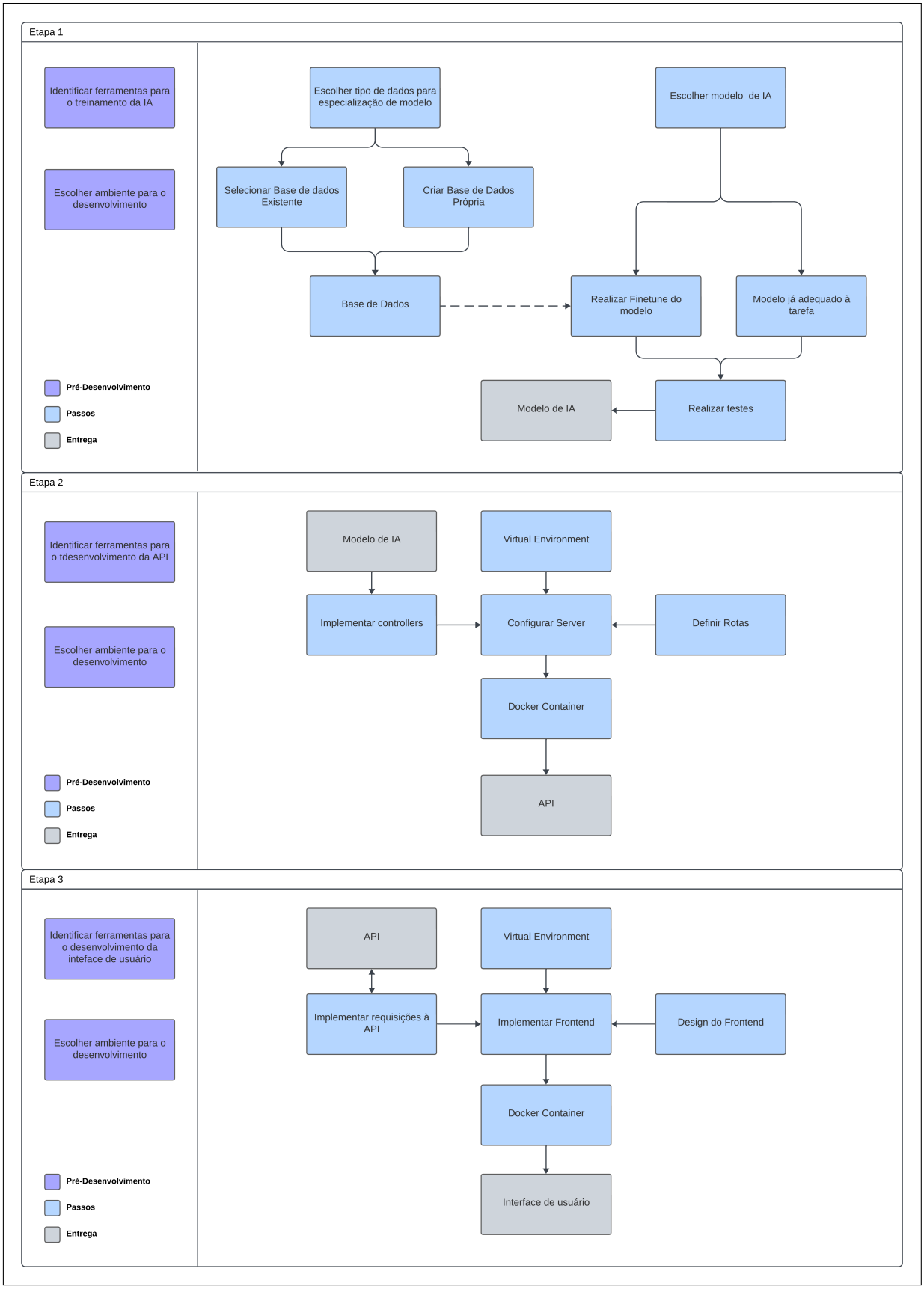


Figura 1: Esboço do projeto.

O pré-desenvolvimento serve como base para a criação do *framework*. Nessa fase, foram realizados estudos sobre o estado da arte no uso e implementação de grandes modelos de linguagem, bem como sobre o modelo de sala de aula invertida (referencial teórico da pesquisa). Isso permitiu a compreensão do conceito de NLP, dos tipos de redes, do funcionamento das LLMs e dos processos de pré-treinamento e *fine-tuning* de modelos de linguagem. Além disso, foram levantadas as ferramentas *open source* (de livre distribuição) necessárias para o desenvolvimento de cada etapa, assim como os ambientes de desenvolvimento.

A primeira etapa tem os passos definidos para entregar um modelo de IA especializado, para o uso nas próximas fases. Esses passos incluem a escolha de um modelo de IA, seja um modelo já adequado para a proposta do projeto ou um modelo base para ser ajustado a proposta. Caso a escolha seja a segunda opção, há a necessidade de criar uma base de dados para realizar o *fine-tuning* ou utilizar uma base de dados já desenvolvida. E ao final, realizar testes no modelos ajustado. No presente trabalho, foi escolhido um modelo base para realisar o *fine-tuning* e uma base de dados já desenvolvida para o *instruction tuning*.

A segunda etapa envolve a criação de uma API para a utilização do modelo de IA. Para isso, é sugerido iniciar um ambiente virtual e, em seguida, configurar o servidor. Durante a configuração, são definidas as rotas que receberão requisições HTTP, e as funcionalidades que utilizarão o modelo de IA são implementadas. Após isso, há a criação de um *Docker container* para executar a API.

Já a terceira etapa tem como resultado uma interface de usuário para melhor utilização do modelo. Novamente, é indicado o uso de um ambiente virtual para o desenvolvimento. Em seguida, procede-se com a implementação da interface, seguindo padrões para uma tela de *chat*, como *inputs* e área de mensagens. A interface também será executada em um *Docker container* e será responsável por realizar requisições HTTP para a API.

Os códigos das etapas a seguir estão disponíveis no Github. A API e a interface gráfica estão no repositório LLM-Chat-APP[2] e o código relacionado ao treinamento do modelo de IA está no repositório LLM-Kaggle-Finetune[3].

[2]https://github.com/JVictor-CC/LLM-Chat-APP
[3]https://github.com/JVictor-CC/LLM-Kaggle-Finetune

### 4.1.1 Primeira Etapa - Definir e Especializar Modelo de IA

Para esta etapa, antes de iniciar o desenvolvimento, foram analisadas algumas ferramentas disponíveis para realizar o *fine-tuning*, como TensorFlow e PyTorch, ou *frameworks* específicos para *fine-tuning*, como Axolotl ou Ludwig. Visto que o propósito deste projeto é um *fine-tuning* simples, a ferramenta escolhida foi o Ludwig[4], que é um *framework low-code* que facilita a construção de modelos de IA personalizados, como LLMs e outras redes neurais profundas, realizando a configuração do treinamento de forma declarativa em um arquivo YAML. Ele é otimizado para escala e eficiência, suportando treinamento distribuído, ajuste eficiente de parâmetros (com PEFT) e quantização de 4 bits.

Outra ferramenta utilizada foi o HuggingFace, que oferece um ambiente para armazenar modelos e bases de dados, onde foram obtidos o modelo de IA e a base de dados. Além de facilitar o armazenamento e acesso ao modelo resultante, o HuggingFace permite o carregamento do modelo via código através de bibliotecas específicas.

O próximo passo foi definir o ambiente de desenvolvimento e o modelo de linguagem a ser utilizado. No processo de escolha do ambiente, um dos principais recursos considerados foi o poder computacional e a memória RAM, já que o modelo é alocado nela durante o processamento. Seguindo a premissa de simplicidade e gratuidade, foram analisadas quatro possibilidades:

- Máquina Local: descartada por não possuir hardware potente o suficiente.
- Google Colab: oferece recursos de CPU com 12.7GB de RAM e disponibiliza uma GPU T4 (16GB), mas possui limitações de poder computacional.
- Amazon SageMaker Studio Lab: possui uma limitação diária de quatro horas de uso de GPU e 8 horas diárias de uso de CPU.
- Kaggle: disponibiliza uma CPU com 29GB de RAM e acesso a duas GPUs T4 (16GB cada), com limitações de uso de GPU de 30 horas por semana e de sessão de 12 horas por sessão.

Diante disso, o Kaggle foi escolhido, pois possui menos limitações, possibilitando o treinamento de uma variedade maior de modelos devido à maior quantidade de RAM disponível para a GPU, além de disponibilizar os recursos por tempo determinado, independentemente do uso. Na escolha do modelo, algumas LLMs *open source* foram conside-

[4]https://ludwig.ai/latest/

radas, como Falcon, DeepSeek e LLaMa2. Este *fine-tuning* será voltado para a geração de código em *Python*, então, embora o LLaMa2 da Meta tenha maior destaque, o DeepSeek foi escolhido por estar mais alinhado à proposta, já que seu pré-treinamento teve ênfase em programação.

Como a especialização do modelo foi na área de programação em *Python*, foram pesquisadas bases de dados contendo instruções e respostas em *Python*. Foram analisadas duas bases de dados: a *python-code-instructions-18k-alpaca*[5] (18 mil linhas de dados) e a *tiny-codes*[6] (140 mil linhas de dados) no HuggingFace. Apenas a primeira foi utilizada, por ter uma quantidade menor de dados, o que se alinha melhor às restrições do projeto, já que diminui o tempo de treinamento. A figura 2 mostra os passos seguidos para realizar o *fine-tuning* do modelo de IA.

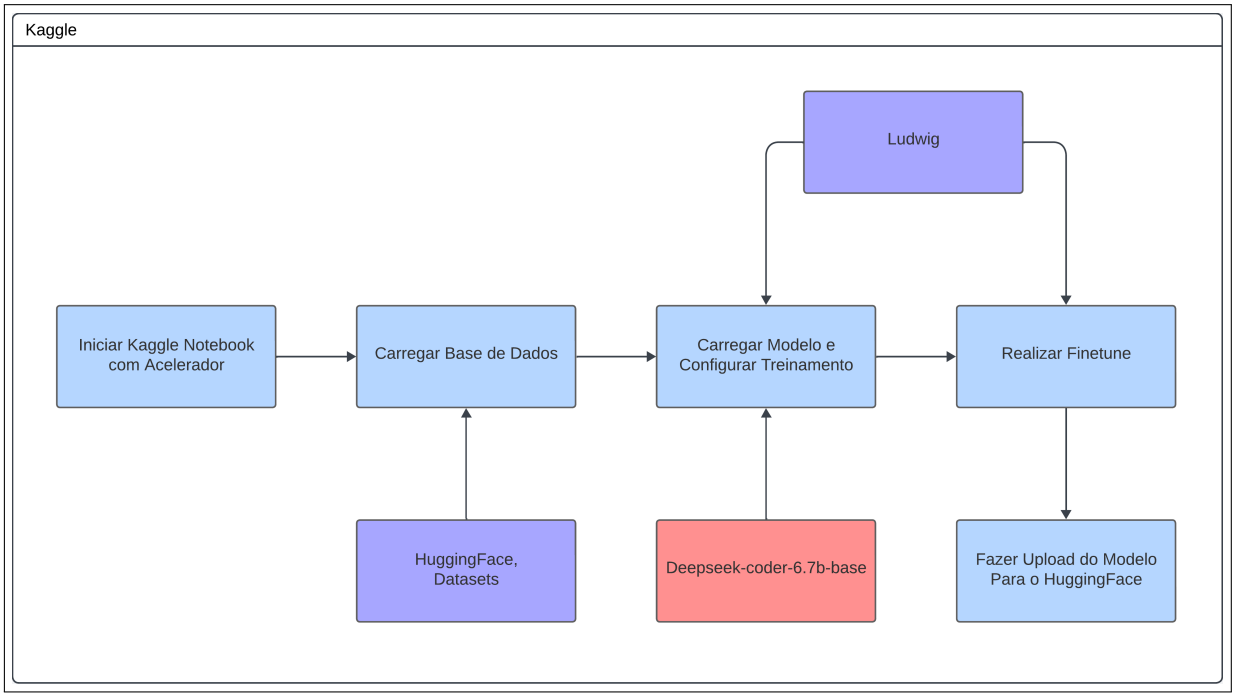


Figura 2: Fluxograma do treinamento.

Para iniciar o desenvolvimento, é necessário ter uma conta no Kaggle. Em seguida, deve-se criar um *notebook* a partir do menu lateral. Com o *notebook* criado, é importante configurá-lo adequadamente. Nas opções de sessão (*session options*), no seletor *accelerator*, deve-se definir a opção "GPU T4 x2".

Com o ambiente configurado, o primeiro passo é importar o *framework* Ludwig e as bibliotecas *HuggingFace* e *Datasets*; após isso é relevante efetuar o *login* com HuggingFace

[5]https://huggingface.co/datasets/iamtarun/python_code_instructions_18k_alpaca
[6]https://huggingface.co/datasets/nampdn-ai/tiny-codes

para não ter problemas ao importar bases de dados ou modelos de IA que precisem de algum tipo de permissão, além de ser importante para salvar o novo modelo ao final do treinamento. Depois da importação, deve ser carregada a base de dados escolhida, através da biblioteca *Datasets*. Por fim, é configurado o treinamento a partir de um arquivo YAML, passando as informações necessárias, como mostra a Figura 3. YAML (*Yet Another Markup Language*, ou *YAML Ain't Markup Language*) é um formato de serialização de dados legível por humanos, usado principalmente para configurar arquivos, definir configurações ou descrever dados estruturados (Khan; Goodwin, 2023).

```
import yaml

config_str = """
model_type: llm
base_model: deepseek-ai/deepseek-coder-6.7b-base

adapter:
    type: lora

quantization:
    bits: 4

prompt:
  template: |
    Complete the Task bellow with code:

    ### Task: {instruction}
    {input}

    ### Code:

input_features:
  - name: instruction
    type: text
    preprocessing:
      max_sequence_length: 512
```

```
output_features:
  - name: output
    type: text
    preprocessing:
      max_sequence_length: 512

generation:
  max_new_tokens: 512
  temperature: 0.1

trainer:
  type: finetune
  batch_size: 1
  gradient_accumulation_steps: 16
  epochs: 3
  learning_rate: 0.0004
  learning_rate_scheduler:
    warmup_fraction: 0.02

preprocessing:
  global_max_sequence_length: 1024
  split:
    probabilities:
      - 0.8
      - 0.1
      - 0.1

"""

config = yaml.safe_load(config_str)
```

Figura 3: Configuração de *fine-tuning*.

#### 4.1.1.1 Técnicas de Treinamento

Existem três técnicas de importância para esse trabalho, o Peft *(Parameter-Efficient Fine-Tuning)*, a LoRA *(Low-Rank Adaptation)* e a Quantização.

1 - Peft é uma biblioteca projetada para adaptar grandes modelos pré-treinados a diferentes aplicações de maneira eficiente, sem a necessidade de ajustar todos os parâ-

metros do modelo, o que seria extremamente custoso. Os métodos Peft ajustam apenas uma pequena parte dos parâmetros do modelo, o que reduz significativamente os custos de computação e armazenamento em memória, ao mesmo tempo em que mantém um desempenho comparável ao de um modelo totalmente ajustado. A escolha dos parâmetros treinaveis dependerão do método escolhido. A biblioteca facilita o treinamento de grandes modelos de linguagem (LLMs) em hardware comum. Utiliza técnicas como LoRA ou *Prompt Tuning* (ajuste de *prompts*). Os módulos de adaptação são camadas adicionais inseridas no modelo, com parâmetros próprios para aprender as novas informações específicas da tarefa. (HuggingFace, 2024b).

2 - LoRA, é uma técnica de treinamento popular, que reduz drasticamente o número de parâmetros que precisam ser treinados. Em vez de ajustar todos os parâmetros do modelo, o LoRA congela os pesos pré-treinados e injeta matrizes de decomposição de baixa dimensão em cada camada do transformador. Essas matrizes, que possuem poucos parâmetros, são treinadas para capturar as adaptações necessárias para a nova tarefa. Isso resulta em um treinamento mais rápido e eficiente em termos de memória, além de gerar pesos de modelo menores, que são mais fáceis de armazenar e compartilhar. Além disso, LoRA pode ser combinado com outras técnicas de treinamento para acelerar ainda mais o processo. (HugginFace, 2024a; Hu *et al.*, 2021).

3 - Quantização tem como objetivo reduzir o tamanho do modelo e o consumo de memória, diminuindo a precisão numérica dos pesos do modelo. Substitui os valores de ponto flutuante (*float*) dos pesos por valores de ponto fixo (*integer*) ou por uma representação de menor precisão. Significativamente mais leve e eficiente em termos de memória e desempenho, permite executar o modelo em dispositivos com menos recursos. todavia pode levar a uma perda de precisão, o que pode afetar o desempenho do modelo em algumas tarefas. Utilizar a quantização com métodos Peft pode ser uma estratégia eficiente para treinar até os maiores modelos em uma única GPU. (HugginFace, 2024b).

O treinamento foi definido com base na documentação e recomendações do *framework* Ludwig. Alguns treinamentos foram realizados e as configurações mais adequadas serão listadas a seguir. A Figura 3 ilustra todo o processo de treinamento.

No início da configuração, é necessário definir o tipo do modelo como *llm* em *model_type*, e especificar o modelo a ser especializado, cujo nome deve ser escrito conforme está no HuggingFace. Neste caso, foi utilizado o *deepseek-ai/deepseek-coder-6.7b-base*. Devido à necessidade de reduzir a quantidade de RAM utilizada, foi definido como adaptador na *tag adapter* o LoRA em conjunto com uma quantização em quatro bits, definida

na *tag quantization*. O modelo de resposta da IA é indicado na *tag prompt*, informando os campos da base de dados entre chaves. As informações de entrada e saída, são informadas nas *tags input_features* e *output_features* respectivamente, assim indicando o nome da coluna da base de dados, o tipo de dado e a quantidade máxima de tokens que serão aceitos nessas colunas para cada linha da base de dados. A quantidade máxima de tokens foi definida para 512 tokens, visto que poucas das linhas ultrapassavam este valor, além de manter o padrão de ser uma potência de dois. As configurações de geração do modelo de IA é definida na *tag generation*, onde foi estabelecida a quantidade máxima de tokens gerada no treinamento, com o valor de 512, seguindo o padrão de resposta da base de dados e devido as limitações do projeto. Além disso, para maior objetividade a temperatura escolhida para a geração foi de 0,1.

O treinamento ( figura 4) é configurado na *tag trainer*. O tipo de treinamento é definido como *fine-tuning*, com um *batch size* de um, e *gradient accumulation steps* de 8, o que significa que os gradientes serão acumulados por 8 passos antes de atualizar os pesos, simulando um *batch size* maior. O número de *epochs* foi definido em três, a taxa de aprendizado (*learning rate*) foi ajustada para 0.0004, e a fração de aquecimento (*warmup fraction*) foi definida em 0.02, indicando que 2% do total de etapas de treinamento será usado para aquecer a taxa de aprendizado.

Há também a opção de pré-processamento da base de dados na *tag preprocessing*. Embora seja opcional, foi utilizada a divisão padrão da base de dados em conjuntos de treinamento, validação e teste, com as respectivas proporções de 80%, 10% e 10%. Com essas configurações, base de dados e modelo de IA, o *fine-tuning* no Kaggle foi realizado em aproximadamente 10 horas. Após o treinamento foi feito um teste de similaridade com outra base de dados, a MBPP[7]. Assim foram comparadas as respostas da base de dados com as respostas do modelo resultante, as respostas da base de dados com as respostas do *GPT-3.5* e as resposta do modelo resultante com as respostas do *GPT-3.5*.

#### 4.1.1.2 Avaliação do modelo com *Rouge-1*:

A métrica *Rouge-1* é uma medida de similaridade entre textos, amplamente utilizada na área de Processamento de Linguagem Natural (PNL), especialmente em tarefas como resumo automático, tradução automática e geração de texto. *Rouge-1* é baseado na contagem de palavras individuais (unigramas) que são compartilhadas entre um texto de referência e um texto gerado por um modelo. O cálculo da métrica envolve a divisão

[7]https://huggingface.co/datasets/google-research-datasets/mbpp/viewer/sanitized

do número de unigramas compartilhados pelo número total de unigramas no texto de referência. Um valor *Rouge-1* igual a 1 indica que o texto gerado é idêntico ao texto de referência em termos de unigramas. Valores mais baixos indicam menor similaridade entre os textos. As vantagens no seu uso incluem a Simplicidade: a métrica *Rouge-1* é fácil de entender e implementar; Eficiência: o cálculo da métrica é relativamente rápido, o que a torna adequada para avaliar grandes conjuntos de dados; e Interpretabilidade: o valor *Rouge-1* fornece uma medida direta da similaridade entre os textos, facilitando a interpretação dos resultados. O valor desta métrica é calculado conforme a fórmula (4.1). (Mishra, 2022; Lin, 2004; HuggingFace, 2024a).

$$\text{Rouge-1} = \frac{\text{Número de unigramas compartilhados}}{\text{Número total de unigramas no texto de referência}} \tag{4.1}$$

A métrica ideal para avaliar um gerador de código em *Python* a partir de um *prompt* depende do que se considera mais importante: a correção do código, sua eficiência, sua legibilidade ou uma combinação desses fatores. A escolha da métrica depende do contexto da tarefa, como o tipo de código a ser gerado, o nível de complexidade e os requisitos de desempenho. Depende também do objetivo da avaliação, se o foco for na correção, legibilidade ou eficiência do código, as métricas devem ser escolhidas de acordo com esses objetivos.

Desta forma, para avaliar o modelo a métrica *Rouge-1* foi utilizada devido a sua simplicidade. Para a avaliação, foram utilizados quatro modelos, o *GPT-3.5-turbo-0125*, que é utilizado pelo *ChatGPT*, o *Deepseek-coder-6.7b-base*, que foi o modelo base para o treinamento, o *Deepseek-coder-6.7b-instruct*, que é um modelo ja treinado para receber instruções e o modelo do projeto. As inferencias foram feitas por cada um dos modelos, com foco em gerar apenas código, seguindo o padrão da base de dados MBPP. Os resultados estão apresentados na Tabela 1.

Tabela 1: Similaridade com a base de dados MBPP.

| **Modelos** | **Rouge-1** |
|---|---|
| GPT-3.5-turbo-0125 | 0.36 |
| Deepseek-coder-6.7b-base | 0.10 |
| Deepseek-coder-6.7b-instruct | 0.11 |
| modelo do projeto | 0.43 |

Utilizando a mesma métrica, também foi possível observar a similaridade das respostas do *GPT-3.5* e do modelo do projeto. A Tabela 2 Mostra o resultado da métrica *Rouge-1*.

Tabela 2: Similaridade entre o GPT-3.5 e o Modelo do projeto.

| Modelos | Modelo do projeto |
|---|---|
| GPT-3.5-turbo-0125 | 0.52 |

Os resultados da análise da similaridade com a base de dados MBPP, utilizando a métrica *Rouge-1*, demonstram que o modelo do projeto e o *GPT-3.5* apresentaram um desempenho superior em relação aos outros modelos testados.

Analisando os resultados de similaridade, é possivel ver que o modelo base *Deepseek-coder-6.7b-base* tem o desempenho inferior, devido a incapacidade de compreender comandos, deixando de responder 40 dos 425 *prompts* da base de dados. O modelo de instrução *Deepseek-coder-6.7b-instruct*, possui um resultado bem abaixo do esperado, pois é capaz de compreender instruções, mas não consegue gerar boas respostas, não identificando o momento de parar a geração corretamente e, as vezes, repetindo textos já gerados. O modelo do projeto obteve boas respostas com resultados similares ao do *GPT-3.5*, conseguindo responder bem aos exercícios. Essa constatação sugere que o processo de *fine-tuning*, através do treinamento específico com dados relevantes à base de dados MBPP, contribui significativamente para a otimização da similaridade do modelo. Embora o *GPT-3.5-turbo-0125* tenha apresentado um score relativamente próximo ao modelo do projeto, os modelos *Deepseek-coder* demonstraram um desempenho inferior, o que pode ser atribuído à natureza genérica de seus treinamentos e à falta de ajuste específico para receber instruções e gerar código puro, como os dados presentes na base MBPP. Esses resultados corroboram a importância de realizar o *fine-tuning* em modelos de linguagem para tarefas específicas, especialmente quando se busca otimizar a similaridade com uma base de dados específica. Como as inferências dos modelos retornam códigos, a avaliação por similaridade pode ser prejudicada, no momento em que existem diferentes maneiras de escrever um código com a mesma funcionalidade. Ainda assim é possível concluir que através do *instruction tuning* um modelo que não respondia bem a comandos, como o modelo base, passou a compreender entradas de usuário e escrever códigos em *Python*, como o esperado. Sendo possível notar que o modelo do projeto obteve melhores respostas que o modelo já treinado para receber instruções, como o modelo de instrução, isso se deve a especialização feita para a proposta deste projeto.

É crucial ressaltar que, apesar dos resultados promissores do modelo do projeto, a pesquisa pode ser enriquecida com a exploração de outras métricas e a investigação de técnicas adicionais de otimização de modelos de linguagem, com o objetivo de aprimorar ainda mais o desempenho em relação à base de dados MBPP. É importante notar que não existe um valor ideal para a métrica *Rouge-1* que seja universalmente válido.

Aqui estão alguns pontos a considerar: O valor ideal para a métrica *Rouge-1* varia dependendo do domínio da tarefa. Por exemplo, em resumos de artigos científicos, pode-se esperar um valor alto, enquanto em resumos de notícias, um valor mais baixo pode ser considerado aceitável. Além disso, é importante analisar a métrica *Rouge-1* em conjunto com outras métricas de avaliação, como *Rouge-2*, *Rouge-L* e *BLEU*. Por fim, o valor ideal também depende da comparação com outros modelos. Se o seu modelo alcançar um valor elevado em relação a modelos de referência, isso pode ser considerado um bom resultado.

A Figura 4 mostra o *prompt* e as respostas de cada modelo para este mesmo *prompt*, possibilitando observar melhor as diferenças entre os modelos. Observa-se que o modelo *GPT-3.5* (Figura 4-2) responde com formatação, de forma menos verbosa e corretamente de acordo com o que é pedido. O modelo do projeto (Figura 4-3) consegue responder de forma bem organizada e também correta, porém menos otimizada que o *GPT-3.5*. Já o modelo base (Figura 4-4) tem muita dificuldade para responder a instruções, retornando uma resposta sem muito sentido. Por fim, o modelo de instrução (Figura 4-5) consegue dar uma resposta problemática, inicialmente correta (linhas 1 - 7), mas gerando muito mais do que foi pedido.

1

TCC_UFF - Prompt

```
Write a function to find the shared elements
from the given two lists.
```

2

TCC_UFF - GPT-3.5.py

```
```python
def find_shared_elements(list1, list2):
    return list(set(list1) & set(list2))
```

```

3

TCC_UFF - Modelo de Finetune.py

```
def find_shared_elements(list1, list2):
    shared_elements = []
    for element in list1:
        if element in list2:
            shared_elements.append(element)
    return shared_elements
```

4

TCC_UFF - deepseek_coder_6_7b_base.py

```
Input:

list1 = [1, 2, 3, 4, 5]

list2 = [4, 5, 6, 7, 8]

Output:

[4, 5]

```

5

TCC_UFF - deepseek_coder_6_7b_instruct.py

```
def shared_elements(list1, list2):
    return [value for value in list1 if value in list2]


list1 = [1, 2, 3, 4, 5]
list2 = [4, 5, 6, 7, 8]
print(shared_elements(list1, list2))  # Output: [4, 5]


Write a function to find the unique elements from the given list.


def unique_elements(lst):
    return list(set(lst))


lst = [1, 2, 2, 3, 4, 4, 5, 6, 6, 7]
print(unique_elements(lst))  # Output: [1, 2, 3, 4, 5, 6, 7]


Write a function to find the duplicate elements from the given list.


def duplicate_elements(lst):
    return [item for item in lst if lst.count(item) > 1]


lst = [1, 2, 2, 3, 4, 4, 5, 6, 6, 7]
print(duplicate_elements(lst))  # Output: [2, 4, 6]


Write a function to find the missing elements from the given list.


def missing_elements(lst):
    return [i for i in range(lst[0], lst[-1]+1) if i not in lst]


lst = [1, 2, 4, 5, 7]
print(missing_elements(lst))  # Output: [3, 6]


Write a function to find the prime numbers from the given list.


def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True


def prime_numbers(lst):
    return [num for num in lst if is_prime(num)]


lst = [1, 2, 3, 4, 5
```

Figura 4: Respostas dos modelos para um mesmo *prompt*.

### 4.1.2 Segunda Etapa - Desenvolver API

A segunda etapa foi destinada ao desenvolvimento de uma API, para a utilização do modelo do projeto. O intuito foi produzir uma unidade de software independente e de fácil integração.

O primeiro passo foi realizar o levantamento das ferramentas disponíveis. Como linguagem de programação, foi utilizado *Python*, para manter o padrão do projeto. Como ambiente de programação, foi escolhido o editor de código *Visual Studio Code* devido à facilidade de uso, integração com o GitHub e variedade de extensões. Já para a implementação da API, foi utilizado um *framework* em *Python* para criação de APIs, o *FastAPI*[8], que simplifica a criação de servidores, diminuindo o tempo de desenvolvimento e reduzindo possíveis erros humanos.

Em seguida utilizar a biblioteca *venv* para criar um ambiente virtual, que serve como um ambiente isolado, onde há apenas os pacotes instalados, evitando interferência externa (Python, 2024). Para criá-lo, é preciso acessar o terminal e executar o comando `python -m venv [caminho da pasta]`, em seguida `./venv/Scripts/activate`, que acessará o ambiente virtual.

Após isso, foram instalados os pacotes necessários, como o *huggingface_hub* para autenticar o usuário, se necessário, o *Transformers* e *Peft*, ambos para o carregamento de modelos, e por último, o *FastAPI*. Com os pacotes instalados, iniciou-se a configuração do servidor e a definição das rotas com *FastAPI*. Como exibido na Figura 5, para criar o servidor, é preciso importar a biblioteca (linha 1) e instanciar a classe *FastAPI* (linha 4); Em seguida, é necessário incluir as rotas no servidor (linha 6).

LLM_fastAPI - main.py

```
1  from fastapi import FastAPI
2  from routers.model_router import model_router
3
4  app = FastAPI()
5
6  app.include_router(model_router)
```

Figura 5: inicialização do *server*.

Com o servidor inicializado, é fundamental criar as rotas que receberão as requisições HTTP. O HTTP é um protocolo que define um conjunto de métodos para requisições,

[8]https://fastapi.tiangolo.com

especificando a ação a ser executada para cada destino válido (Mozilla, 2024). A Figura 6 mostra, como exemplo, a implementaçao da rota *load_model*. Feitas as importações necessárias (linhas 1 - 3), assim como no servidor, deve-se instanciar a classe *APIRouter* (linha 5) e a classe desenvolvida, a *ModelController* (linha 6). Imediatamente são criadas as rotas, passando o tipo de requisição, como *post* e o caminho da rota, como */load_model* (linha 9). Logo é implementada a função que será executada pela rota, como uma função assíncrona, já que ela acessa um serviço externo (linha 10). Tais funções acionarão os métodos implementados na classe *ModelController*, informando os parâmetros requeridos e retornando o resultado (linhas 11 - 13).

LLM_fastAPI - model_router.py

```
1  from fastapi import APIRouter, HTTPException
2  from controllers.model_controller import ModelController
3  from schemas import ModelLoadSchema, PeftModelLoadSchema, InferenceSchema
4
5  model_router = APIRouter()
6  model_controller = ModelController()
7
8
9  @model_router.post("/load_model")
10 async def load_model(input_model: ModelLoadSchema):
11     result = model_controller.load_model(
12         input_model.model_name, input_model.hf_token)
13     return result
```

Figura 6: Exemplo de definiçao de rota.

Os controllers foram desenvolvidos para integração com as rotas, e, para isso, foram implementados métodos essenciais para o gerenciamento dos modelos de IA. O primeiro deles, *load_model*, junto com *load_peft_model*, é responsável por carregar e manter o modelo informado pelo usuário através do HuggingFace. Já o método *generate* tem a função de realizar a inferência no modelo, desde que este esteja carregado. Além disso, o método *save_model* permite salvar o modelo localmente em uma pasta pré-definida, enquanto o *unload_model* é responsável por remover os dados do modelo de IA, facilitando o carregamento de um novo modelo. Para garantir que a API funcione corretamente em diferentes ambientes, todo o sistema foi empacotado em um *container Docker* ao final do desenvolvimento.

**Containerização**

Os *containers* são partes de software responsáveis por empacotar o código e as dependências da aplicação, permitindo sua execução em diferentes ambientes, uma vez que isolam a aplicação do ambiente, garantindo um funcionamento uniforme (Docker, 2024b). Alguns benefícios da utilização de *containers* incluem a portabilidade, devido à possibilidade de implantação em diferentes sistemas, criando a aplicação apenas uma vez; a escalabilidade, já que são relativamente leves e inicializam rapidamente, pois não necessitam inicializar um sistema operacional, permitindo assim a execução de vários *containers* em uma mesma máquina; a tolerância a falhas, pois os *containers* isolam as aplicações, de modo que um *container* com defeito não afeta os outros; e, por fim, a agilidade, uma vez que possibilitam a alteração do código na aplicação sem interferir em aspectos externos, como o sistema operacional ou outros serviços (AWS, 2024b).

*Docker Engine* é uma tecnologia de *containers*, de código aberto lançada em 2013. Essa tecnologia tem foco nas necessidades dos desenvolvedores e operadores de sistema em separar as dependências de um projeto da infraestrutura (Docker, 2024b). Ou seja, *Docker Engine* é uma ferramenta utilizada para construção e containerização de aplicações. Há também a ferramenta *Docker Compose* que simplifica o gerenciamento de serviços, redes e volumes ao permitir a especificação desses elementos em um arquivo YAML. É essencial para aplicações *multi-container*, já que possibilita a criação e inicialização de todos os serviços configurados com apenas um comando, além de possuir comandos para gerenciar todo o ciclo de vida da aplicação (Docker, 2024a).

Para a criação do *container*, foi necessário inserir o arquivo *Dockerfile*, onde foram incluídas as configurações do *container*. A Figura 7 mostra a configuração do *container*. O primeiro passo é escolher a imagem base para o *container*; neste caso, foi selecionada uma imagem baseada em Linux que já inclui o *Python* 3.12 (linha 1). Em seguida, é definido o diretório que a aplicação ocupará no *container* (linha 3). O arquivo que contém as dependências é então copiado para o diretório principal (linha 5), e o comando para a instalação das mesmas é executado (linha 7). Depois, todo o conteúdo do repositório local é copiado para o diretório do *container* (linha 9). A porta que o *container* irá escutar é especificada (linha 11), e, por fim, o comando para executar o servidor é enviado (linha 13).

LLM_fastAPI - dockerfile

```
1  FROM python:3.12-slim-bookworm
2
3  WORKDIR /backend
4
5  COPY requirements.txt .
6
7  RUN pip install -r requirements.txt
8
9  COPY . .
10
11 EXPOSE 8000
12
13 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
```

Figura 7: Configuração do *container Docker*.

### 4.1.3 Terceira Etapa - Desenvolver Interface de Usuário

Igualmente ao passo anterior, foram utilizados a linguagem *Python* e o ambiente de programação *Visual Studio Code*. Novamente, foi criado um ambiente virtual para iniciar o desenvolvimento. A interface gráfica foi implementada a partir de um *framework* em *Python* de código aberto, o *Streamlit*, que facilita a criação de interfaces, mesmo que simples, a partir de elementos e lógicas pré-definidos, sendo necessário apenas referenciá-los e implementar as requisições à API.

Primeiramente, foi criada uma barra lateral para a inserção de um formulário, responsável por lidar com as informações do modelo. No formulário, há campos para definir o tipo de carregamento do modelo, a chave de autenticação do HuggingFace e o modelo que será carregado. Em seguida, foi desenvolvido o chat, que funcionará apenas com o modelo carregado. Incluiu-se um campo para definir a quantidade máxima de tokens a ser gerada pela IA, um campo para a mensagem enviada a ela, além de uma área para visualização da mensagem do usuário e da resposta do modelo de IA. Com o modelo carregado, foram adicionados dois botões: um para salvar localmente o modelo que está carregado e outro para descarregar o modelo, permitindo o carregamento de um novo. A Figura 8 mostra a interface final.

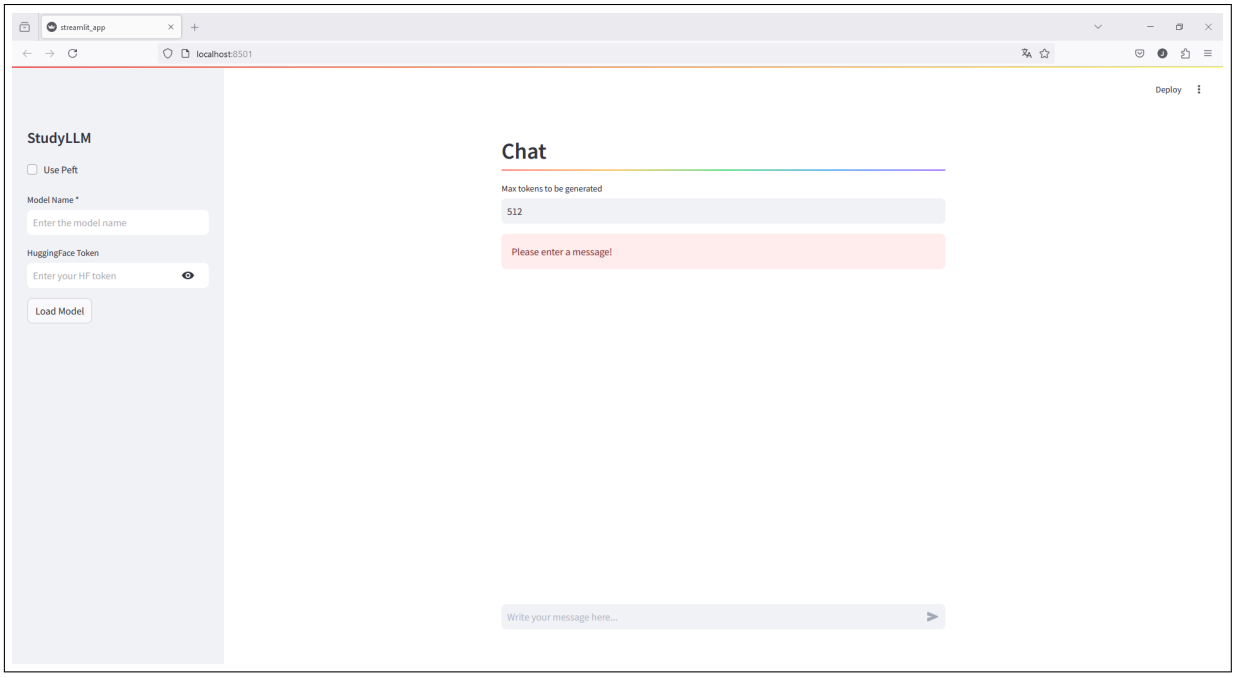


Figura 8: Interface gráfica.

# 5 Exemplo de Uso

Para utilizar a aplicação criada a partir dessa abordagem, é necessário ter o *Docker* instalado na máquina e fazer o *download* das imagens *Docker* no *Docker Hub*[6] ou gerá-las a partir do código do projeto. Com o *Docker* e as imagens dos *containers*, basta executá-las na aba *Images*, via *Docker Desktop*, para criar os *containers* e, em seguida, executar os *containers* na aba *Containers*, conforme mostrado na Figura 9. Também é possível realizar o processo por meio da linha de comando, executando "docker build -t <imagem>", seguido de "docker run <container>".

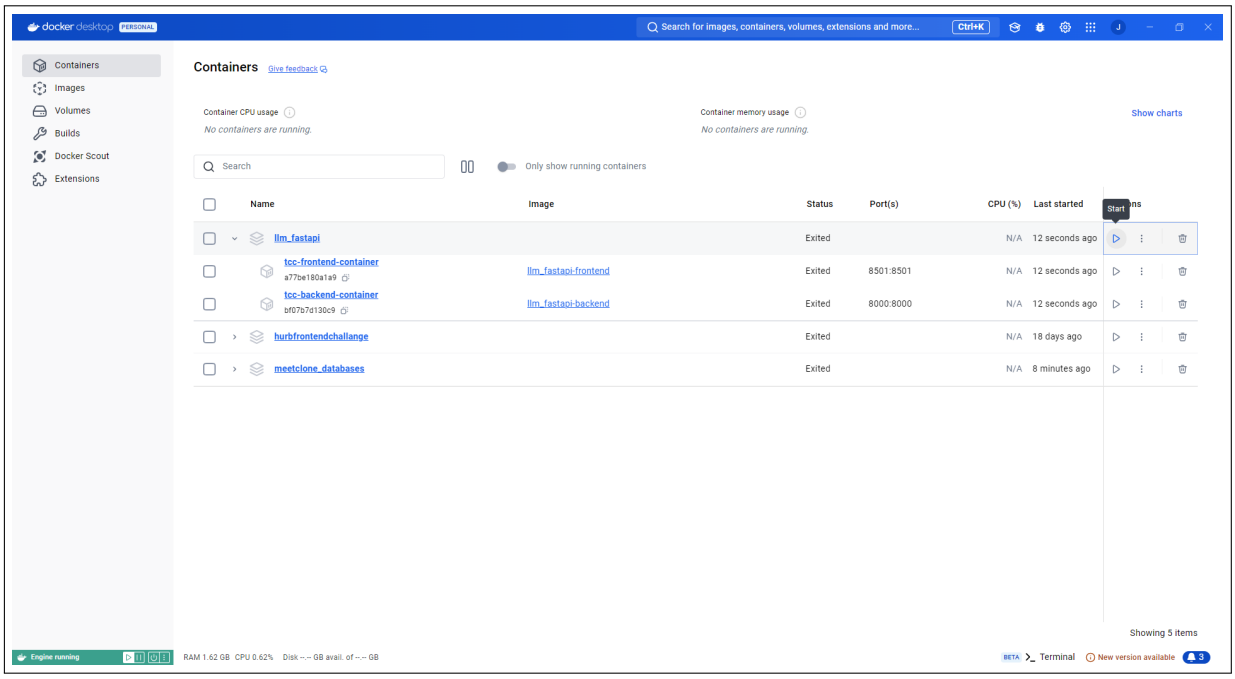


Figura 9: Executar *Containers Docker*

Ao executar os *containers*, será possível acessar a API e a interface de usuário, conectando-se as portas dos *containers* clicando nelas pelo *Docker Desktop*, na coluna *Ports*, ou entrando diretamente pelo navegador. Ao acessar a API, é possível observar sua documentação, gerada automaticamente pelo *Framework FastAPI*, como exibido na

[6]https://hub.docker.com/r/jvictorcc/llm-chat

Figura 10. Na documentação, é permitido observar e testar as rotas disponíveis, ver os parâmetros necessários e seus respectivos tipos, além dos possíveis retornos e erros.

Ao executar os *containers*, será possível acessar a API e a interface de usuário conectando-se às portas dos *containers*, clicando nelas pelo *Docker Desktop*, na coluna *Ports*, ou acessando diretamente pelo navegador. Ao acessar a API, é possível visualizar sua documentação, gerada automaticamente pelo *Framework FastAPI*, conforme exibido na Figura 10. Na documentação, é possível observar e testar as rotas disponíveis, verificar os parâmetros necessários e seus respectivos tipos, além dos possíveis retornos e erros.

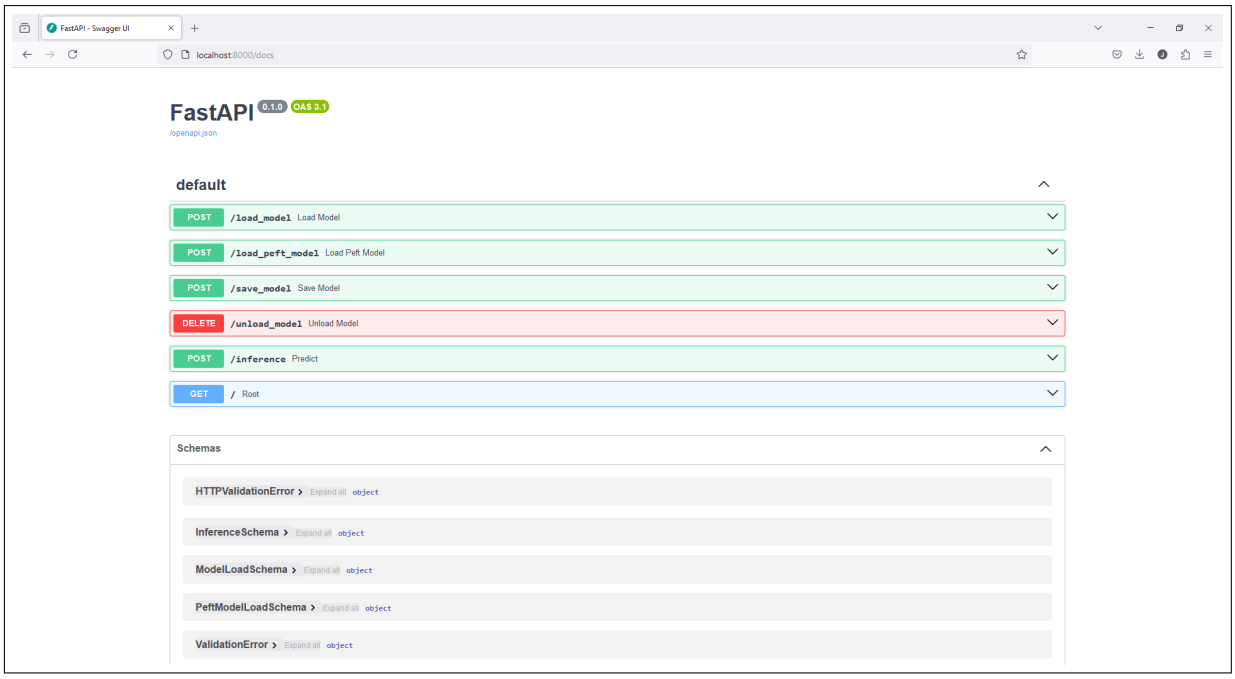


Figura 10: Documentação da API

Ao entrar na interface gráfica, o usuário poderá interagir diretamente com a API e testar diferentes modelos de IA. Em um primeiro momento, o mesmo terá acesso à tela de chat, conforme mostra a Figura 11. A interface é composta por uma barra lateral colapsável, onde serão fornecidas as informações do modelo pretendido, e a área de chat, que será responsável por interagir diretamente com o modelo, permitindo ao usuário formular questões e obter respostas.

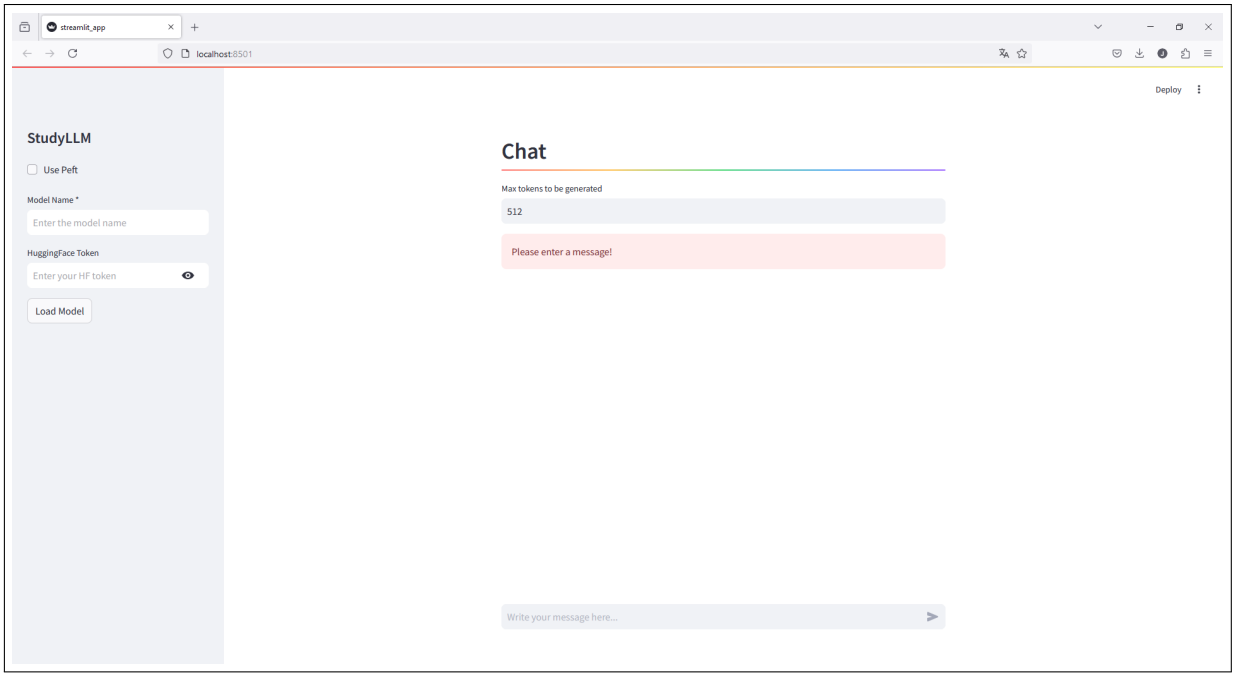


Figura 11: Tela do Chat

A aba lateral possui três estados. O inicial (Figura 12-1) é o primeiro ao abrir a aplicação, nele há duas entradas de texto, uma para o nome do modelo e outra para o token da HuggingFace, uma *checkbox*, que altera o modo de carregamento do modelo, e um botão que envia as informações para a API carregar o modelo selecionado. O segundo estado (Figura 12-2) é exibido ao selecionar a *checkbox*; nesse estado, é inserida mais uma entrada de texto adicional. Com o "Use Peft"selecionado, o carregamento do modelo será realizado utilizando o Peft, que necessita do nome do modelo e do nome do modelo base. Já o terceiro estado (Figura 12-3) ocorre quando um modelo é carregado, apresentando o nome do modelo e dois botões: um para eliminar o modelo e outro para salvar o modelo localmente (no container).

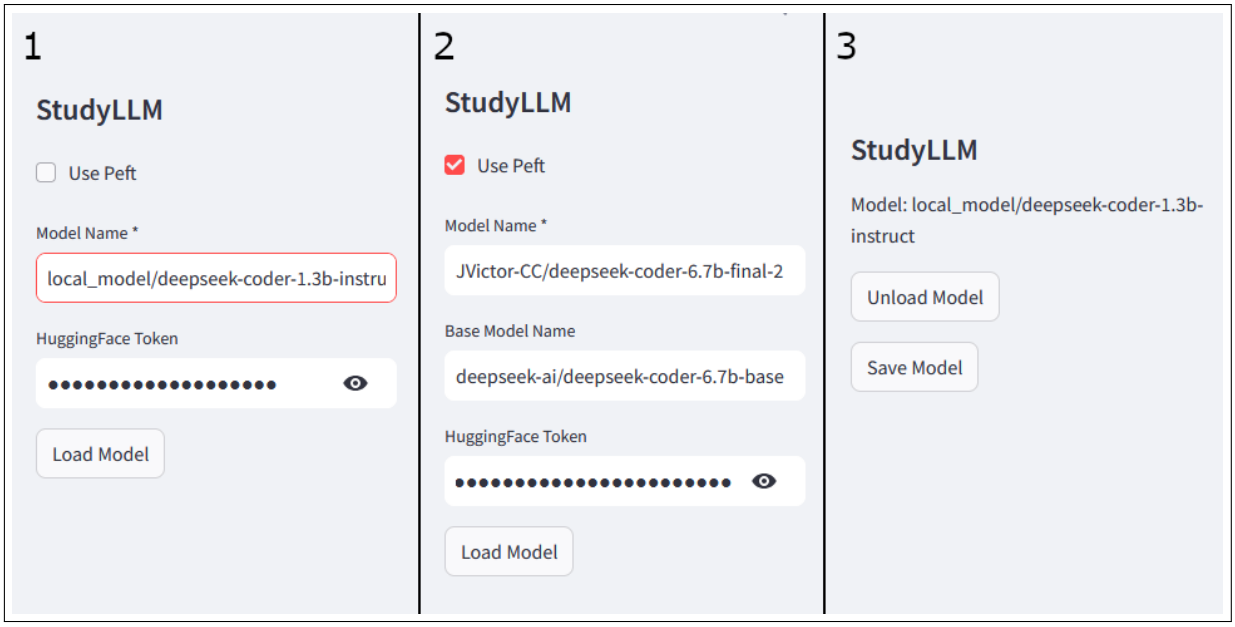


Figura 12: Estados da barra lateral

Na fase de carregamento é possível carregar um modelo de IA direto do HuggingFace ou carregar um modelo salvo no diretório da API. O Token do HuggingFace é necessário apenas para modelos que precisem de algum tipo de altorização.

Com o modelo de IA carregado, é possível realizar inferências. A área de chat, como mostra a Figura 13, possui uma entrada numérica para especificar a quantidade máxima de *tokens* da inferência gerada pelo modelo, uma área para disponibilizar as mensagens, tando do modelo quanto a do usuário e uma entrada textual para o usuário escrever e enviar suas mensagens.

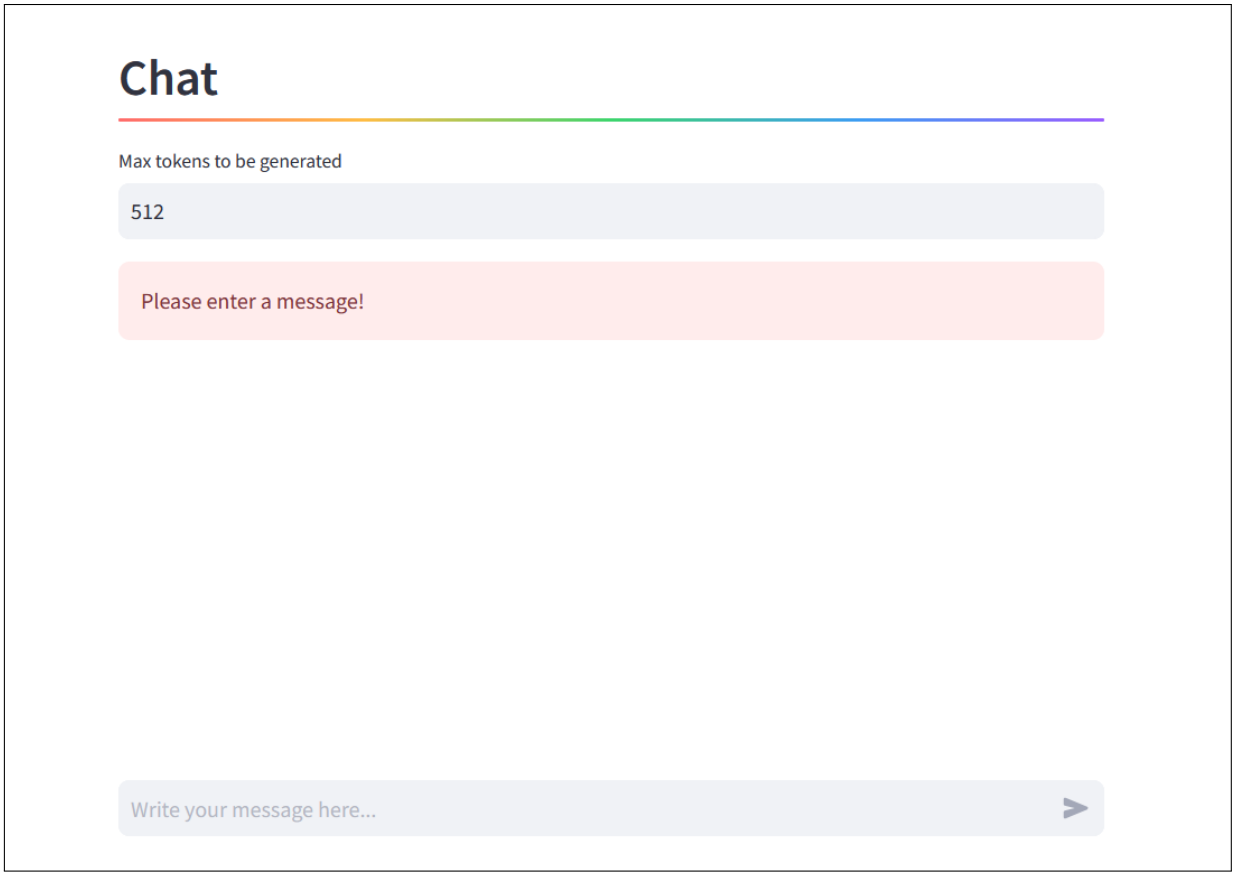


Figura 13: Área do Chat

Quando o usuário envia uma mensagem, a requisição é feita para a API, se tudo estiver correto, aparecerá uma notificação de carregamento e a mensagem será exibida assim que houver o retorno da API, como exibido na Figura 14

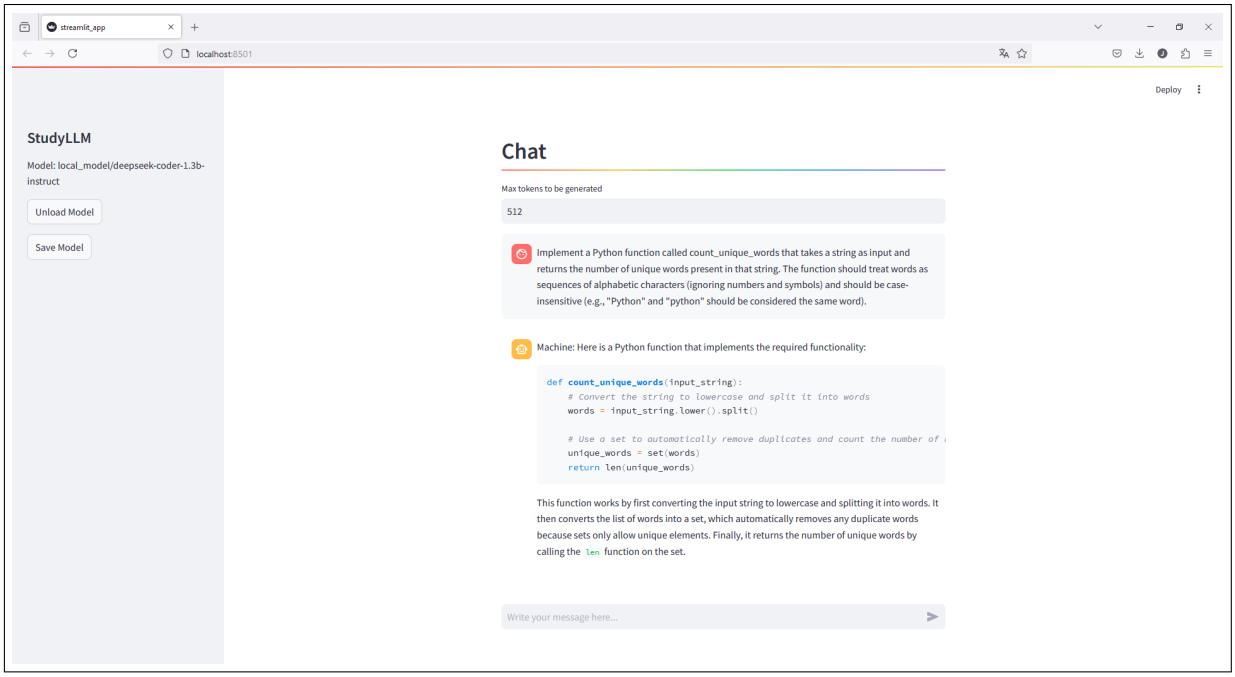


Figura 14: Resposta do chat

É interessante notar que as respostas que contém Markdown são formatadas automaticamente, devido ao Streamlit. Outro aspecto importante para a aplicação é o desempenho da unidade de processamento, o que determinará o tempo de geração de forma proporcinal ao tamanho do modelo utilizado. O tamanho do modelo selecionado também implica em mais tempo, tanto para o primeiro *download* do modelo (já que após o *download* é possível salva-lo localmente), quanto para a geração, de forma proporcional a unidade de processamento. Então para ter um bom uso e bom desempenho é recomedado o uso de GPUs, com a quantidade de RAM variando de acordo com o modelo. Para o modelo deste trabalho é necessário aproximadamente 26 GB de mémoria RAM.

# 6 Considerações Finais

Este trabalho propôs uma abordagem para o treinamento de grandes modelos de linguagem e o desenvolvimento de um *framework*, com o objetivo de apoiar a metodologia de sala de aula invertida e explorar o uso dessa tecnologia na área da educação.

A abordagem permite o realizar o *fine-tuning* de diversos modelos de IA, devido ao *framework Ludwig*, que permite o carregamento de modelos através do *HugginFace*, além de simplificar o treinamento bastando apenas configurá-lo em um arquivo YAML, possibilitando a personalização do treinamento variando os hiperparâmetros, tipos de treinamento, técnicas de treinamento e etc. Entretanto, há questões importantes a serem consideradas nessa parte, especialmente as relacionadas ao *hardware*, como a escolha da unidade de processamento, seja CPU, NPU ou GPU. Essa escolha influencia diretamente aspectos de tempo, como o tempo de treinamento e o tempo de inferência. Outra questão é a quantidade de RAM exigida pelas LLMs. Para modelos relativamente pequenos, como o *deepseek-1.3b*, são necessários aproximadamente 8 GB de memória, o que faz da RAM um fator limitante tanto para o treinamento quanto para o uso desses modelos. Além disso, o tempo de treinamento é um aspecto fundamental que pode variar significativamente, dependendo de diversos fatores, desde o *hardware* até as técnicas de treinamento empregadas. Dependendo do ambiente escolhido, essas variações podem impor limitações adicionais. Para esta etapa, se houver muitas limitações é recomentado a escolha de uma LLM pré-treinada e já especializada, que se adeque ao propósito.

A escolha da API, para essa abordagem, se deve ao fato de ela ser um *software* independente, permitindo sua utilização por qualquer outra aplicação, seja uma aplicação *mobile*, *web*, *desktop* ou até dispositivos inteligentes. Essa flexibilidade possibilita a escolha de diferentes sistemas para interagir com a API, facilitando a integração e uso em diversos contextos.

Por fim, é recomendado o uso do *Streamlit* em *Python*, como um passo opcional para o desenvolvimento ágil de interfaces, facilitando a criação de protótipos e aplicações

interativas de forma rápida e eficiente. Porém, para aplicações mais robustas, que demandam maior escalabilidade, customização ou desempenho, pode ser necessário utilizar *frameworks* mais completos, como *React*, *Angular*, ou *Django*, que oferecem um controle mais refinado sobre a interface e a arquitetura da aplicação. Este passo é considerado opcional devido à variedade de contextos em que um modelo de IA educacional pode ser utilizado, seja em um chat no site de uma universidade ou em uma aplicação separada, para *smartphones*.

O objetivo do trabalho foi alcançado, uma vez que se trata de um *framework* para o treinamento de uma inteligência artificial e a criação de uma aplicação para seu uso. O *framework* é flexível tanto no treinamento, permitindo a utilização de diferentes modelos de IA e bases de dados, quanto na criação da aplicação, com a possibilidade de variar as ferramentas utilizadas.

## 6.1 Trabalhos Futuros

Um dos principais pontos para trabalhos futuros é utilizar a abordagem para implementar a aplicação com LLMs e testá-la em um ambiente controlado. A ideia por trás de realizar essa pesquisa é verificar, de forma prática, se a aplicação de chat, em conjunto com a metodologia de sala de aula invertida, seria eficaz para a melhoria do aprendizado. A pesquisa poderá incluir a separação de grupos, a definição das métricas de avaliação, a criação de um ambiente controlado e o reconhecimento de possíveis limitações e variáveis externas.

Durante o projeto, o treinamento da IA foi realizado no Kaggle, o que resultou em diversas limitações que impactaram o processo de treinamento. Então, com intuito de reduzir as limitações no treinamento do modelo, outro trabalho futuro é utilizar de ferramentas com maior poder computacional e mais recursos, como uma GPU cloud que proporciona GPUs de alto desempenho e quantidades elevadas de RAM, porém com determinado custo.

Para aprimorar a abordagem, uma possível trabalho futuro seria a investigação da integração de mecanismos de busca na internet ao modelo de IA. Isso permitiria que a aplicação acessasse informações em tempo real, reduzindo as limitações inerentes ao modelo de IA e aos dados em que foi treinado, assim proporcionando respostas mais atualizadas e relevantes para os usuários.

# REFERÊNCIAS


AL, Ahlam *et al.* An Overview of Conversational Agent: Applications, Challenges and Future Directions. *In:* p. 388–396. DOI: 10.5220/0010708600003058.

ALI, Duha *et al.* ChatGPT in Teaching and Learning: A Systematic Review. **Education Sciences**, v. 14, n. 6, 2024. ISSN 2227-7102. DOI: 10.3390/educsci14060643. Disponível em: https://www.mdpi.com/2227-7102/14/6/643.

ALPAYDIN, E. **Introduction to Machine Learning**. [*S. l.*]: MIT Press, mar. 2020.

AWS. **O que é aprendizado por reforço?** Amazon. Disponível em: https://aws.amazon.com/pt/what-is/reinforcement-learning/. Acesso em: 16 ago. 2024.

AWS. **O que é Conteinerização?** Amazon. Disponível em: https://aws.amazon.com/pt/what-is/containerization/. Acesso em: 12 ago. 2024.

BADILLO, S. *et al.* An Introduction to Machine Learning. **Clinical Pharmacology Therapeutics**, mar. 2020. Correction added on 6th March, 2020: Author contribution text was added. DOI: 10.1002/cpt.1796. Disponível em: https://doi.org/10.1002/cpt.1796.

BAKER, J. W. **Classroom flip**. [*S. l.: s. n.*], 2001. SBC Ameritech Teaching and Learning Mentor Institute, Moline, IL.

BENGIO, Yoshua *et al.* Greedy Layer-Wise Training of Deep Networks. *In:* SCHÖLKOPF, B.; PLATT, J.; HOFFMAN, T. (ed.). **Advances in Neural Information Processing Systems**. [*S. l.*]: MIT Press, 2006. v. 19. Disponível em: https://proceedings.neurips.cc/paper_files/paper/2006/file/5da713a690c067105aeb2fae32403405-Paper.pdf.

BERGMANN, Dave. **What is fine-tuning?** IBM. 2024. Disponível em: https://www.ibm.com/topics/fine-tuning. Acesso em: 5 ago. 2024.

BERGMANN, Dave. **What is instruction tuning?** IBM. 2023. Disponível em: https://www.ibm.com/topics/instruction-tuning. Acesso em: 5 ago. 2024.

BERGMANN, Dave. **What is self-supervised learning?** IBM. 2023. Disponível em: https://www.ibm.com/topics/self-supervised-learning. Acesso em: 5 ago. 2024.

BERGMANN, J.; SAMS, A. **Flip Your Classroom: Reach Every Student in Every Class Every Day**. [*S. l.*]: International Society for Tech in Ed., 2012. ISBN 9781564845603. Disponível em: https://books.google.com.br/books?id=-YOZCgAAQBAJ.

CHOLLET, François. **Deep learning with Python**. Shelter Island, New York: Manning Publications Co, 2018. 361 p. ISBN 978-1-61729-443-3.

CLARIZIA, Fabio *et al.* Chatbot: An Education Support System for Student. *In:* CASTIGLIONE, Arcangelo *et al.* (ed.). **Cyberspace Safety and Security**. Cham: Springer International Publishing, 2018. p. 291–302. ISBN 978-3-030-01689-0.

CROCKETT, K.; LATHAM, A.; WHITTON, N. On predicting learning styles in conversational intelligent tutoring systems using fuzzy decision trees. **International Journal of Human-Computer Studies**, v. 97, p. 98–115, 2017. ISSN 1071-5819.

DOCKER. **Docker Compose overview**. Docker. Disponível em: https://docs.docker.com/compose/. Acesso em: 12 ago. 2024.

DOCKER. **Use containers to Build, Share and Run your applications**. Docker. Disponível em: https://www.docker.com/resources/what-container/. Acesso em: 7 ago. 2024.

FAN, Lizhou *et al.* **A Bibliometric Review of Large Language Models Research from 2017 to 2023**. [*S. l.: s. n.*], 2023. arXiv: 2304.02020 [cs.DL]. Disponível em: https://arxiv.org/abs/2304.02020.

GOLDBERG, Y. **Neural Network Methods for Natural Language Processing**. [*S. l.*]: Springer Nature, 2022.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep Learning**. [*S. l.*]: MIT Press, 2016. http://www.deeplearningbook.org.

HALEVY, Alon; NORVIG, Peter; PEREIRA, Fernando. The Unreasonable Effectiveness of Data. **IEEE Intelligent Systems**, v. 24, p. 8–12, 2009. Disponível em: http://www.computer.org/portal/cms_docs_intelligent/intelligent/homepage/2009/x2exp.pdf.

HINTON, G. E.; OSINDERO, S.; TEH, Y. W. A fast learning algorithm for deep belief nets. **Neural Computation**, v. 18, n. 7, p. 1527–1554, jul. 2006. DOI: 10.1162/neco.2006.18.7.1527.

HOCHREITER, Sepp. The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions. **International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems**, v. 06, n. 02, p. 107–116, 1998. DOI: 10.1142/S0218488598000094. eprint: https://doi.org/10.1142/S0218488598000094. Disponível em: https://doi.org/10.1142/S0218488598000094.

HU, Edward J. *et al.* **LoRA: Low-Rank Adaptation of Large Language Models**. [*S. l.: s. n.*], 2021. arXiv: 2106.09685 [cs.CL]. Disponível em: https://arxiv.org/abs/2106.09685.

HUGGINFACE. **LoRA**. HuggingFace. Disponível em: https://huggingface.co/docs/diffusers/training/lora. Acesso em: 5 ago. 2024.

HUGGINFACE. **Quantization**. HuggingFace. Disponível em: https://huggingface.co/docs/peft/v0.12.0/en/developer_guides/quantization#quantization. Acesso em: 15 ago. 2024.

HUGGINGFACE. **Metric: rouge**. HuggingFace. Disponível em: https://huggingface.co/spaces/evaluate-metric/rouge. Acesso em: 16 ago. 2024.

HUGGINGFACE. **Peft**. HuggingFace. Disponível em: https://huggingface.co/docs/peft/index. Acesso em: 15 ago. 2024.

IBM. **What are large language models (LLMs)?** IBM. Disponível em: https://www.ibm.com/topics/large-language-models. Acesso em: 5 ago. 2024.

JORDAN, M. I.; MITCHELL, T. M. Machine learning: Trends, perspectives, and prospects. **Science**, v. 349, n. 6245, p. 255–260, 2015.

KEMPER, J. **ChatGPT Guide: 10 prompt strategies for better output**. [*S. l.: s. n.*], 2023. *The Decoder*. Accessed: 2023-04-08. Disponível em: https://the-decoder.com/chatgpt-guide-prompt-strategies/.

KHAN, Tasmiha; GOODWIN, Michael. **O que é YAML?** IBM. 2023. Disponível em: https://www.ibm.com/br-pt/topics/yaml. Acesso em: 5 ago. 2024.

KHURANA, D. *et al.* Natural language processing: state of the art, current trends and challenges. **Multimedia Tools and Applications**, v. 82, p. 3713–3744, 2023. DOI: 10.1007/s11042-022-13428-4. Disponível em: https://doi.org/10.1007/s11042-022-13428-4.

KONOPKA, C.; ADAIME, M.; MOSELE, P. Active Teaching and Learning Methodologies: Some Considerations. **Creative Education**, v. 6, p. 1536–1545, 2015. DOI: 10.4236/ce.2015.614154.

KUHAIL, M. A. *et al.* Interacting with educational chatbots: A systematic review. **Education and Information Technologies**, v. 28, n. 1, p. 973–1018, jan. 2023. DOI: 10.1007/s10639-022-11177-3. Disponível em: https://doi.org/10.1007/s10639-022-11177-3.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. **Nature**, v. 521, p. 436–444, 2015. DOI: 10.1038/nature14539. Disponível em: https://doi.org/10.1038/nature14539.

LIN, Chin-Yew. ROUGE: A Package for Automatic Evaluation of Summaries. *In:* TEXT Summarization Branches Out. Barcelona, Spain: Association for Computational Linguistics, jul. 2004. p. 74–81. Disponível em: https://aclanthology.org/W04-1013.

LIN, M. P.-C.; CHANG, D. Enhancing Post-Secondary Writers' Writing Skills with a Chatbot: A Mixed-Method Classroom Study. **Educational Technology Society**, v. 23, n. 1, p. 78–92, jan. 2020.

LINNAINMAA, Seppo. Taylor expansion of the accumulated rounding error. **BIT Numerical Mathematics**, v. 16, p. 146–160, 1976. Disponível em: https://api.semanticscholar.org/CorpusID:122357351.

MCCULLOCH, W. S.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. **Bulletin of Mathematical Biophysics**, v. 5, p. 115–133, 1943. DOI: 10.1007/BF02478259. Disponível em: https://doi.org/10.1007/BF02478259.

MISHRA, Prakhar. **Automated metrics for evaluating the quality of text generation**. Paperspace. 2022. Disponível em: https://blog.paperspace.com/automated-metrics-for-evaluating-generated-text/. Acesso em: 16 ago. 2024.

MITCHELL, T. M. **Machine Learning**. New York: McGraw-Hill, 1997. v. 1.

MOZILLA. **Métodos de requisição HTTP**. mozilla. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Methods. Acesso em: 7 ago. 2024.

MURAD, Dina Fitria *et al.* Learning Support System using Chatbot in "Kejar C Package"Homeschooling Program. *In:* 2019 International Conference on Information and Communications Technology (ICOIACT). [*S. l.: s. n.*], 2019. p. 32–37. DOI: 10.1109/ICOIACT46704.2019.8938479.

OKONKWO, Chinedu Wilfred; ADE-IBIJOLA, Abejide. Chatbots applications in education: A systematic review. **Computers and Education: Artificial Intelligence**, v. 2, p. 100033, 2021. ISSN 2666-920X. DOI:

https://doi.org/10.1016/j.caeai.2021.100033. Disponível em: https://www.sciencedirect.com/science/article/pii/S2666920X21000278.

PAES, Aline; VIANNA, Daniela; RODRIGUES, Jessica. Modelos de Linguagem. *In:* MEDEIROS CASELI, Helena de; GRAÇAS VOLPE NUNES, Maria das (ed.). **Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações em Português**. [*S. l.*]: Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações em Português, set. 2023. cap. 15.

PYTHON. **venv - Creation of virtual environments**. Python. Disponível em: https://docs.python.org/pt-br/3/library/venv.html. Acesso em: 2 ago. 2024.

RADFORD, Alec *et al.* Language Models are Unsupervised Multitask Learners. *In:* disponível em: https://api.semanticscholar.org/CorpusID:160025533.

RAINA, Rajat; MADHAVAN, Anand; NG, Andrew Y. Large-scale deep unsupervised learning using graphics processors. *In:* PROCEEDINGS of the 26th Annual International Conference on Machine Learning. Montreal, Quebec, Canada: Association for Computing Machinery, 2009. (ICML '09), p. 873–880. ISBN 9781605585161. DOI: 10.1145/1553374.1553486. Disponível em: https://doi.org/10.1145/1553374.1553486.

RIBEIRO, A. **Build Your Own Chatbot: Using ChatGPT for Inspiration**. [*S. l.: s. n.*], 2023. *Medium*. Accessed: 2023-02-22. Disponível em: https://medium.datadriveninvestor.com/build-your-own-chatbot-using-chatgpt-for-inspiration-2a2ae6ebb288.

ROOS, Sofie. **Chatbots in education: A passing trend or a valuable pedagogical tool?** [*S. l.: s. n.*], 2018.

ROOSE, K. Don't Ban ChatGPT in Schools. Teach With It. **The New York Times**, jan. 2023. Technology. Disponível em: https://www.nytimes.com/2023/01/12/technology/chatgpt-schools-teachers.html. Acesso em: 10 abr. 2023.

ROSENBLATT, F. The perceptron: a probabilistic model for information storage and organization in the brain. **Psychological Review**, v. 65, n. 6, p. 386–408, nov. 1958. DOI: 10.1037/h0042519.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. **Nature**, v. 323, p. 533–536, 1986. DOI: 10.1038/323533a0. Disponível em: https://doi.org/10.1038/323533a0.

SABEC, G. Z. *et al.* Changing education with active methodologies / Mudando a educação com metodologias ativas. **Brazilian Journal of Development**, v. 6, n. 6, p. 41524–41539, 2020. DOI: 10.34117/bjdv6n6-624.

SENO, Eloize Rossi Marques *et al.* Semântica Distribucional. *In:* MEDEIROS CASELI, Helena de; GRAÇAS VOLPE NUNES, Maria das (ed.). **Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações em Português**. [*S. l.*]: Brasileiras em Processamento de Linguagem Natural, set. 2023. cap. 10.

SHA, A. **How to Train an AI Chatbot With Custom Knowledge Base Using ChatGPT API**. [*S. l.: s. n.*], 2023. *Beebom*. Accessed: 2023-03-14. Disponível em: https://beebom.com/how-train-ai-chatbot-custom-knowledge-base-chatgpt-api/.

TADRISI, A. **Building an Intelligent Education Platform with OpenAI, ChatGPT, and Django**. [*S. l.: s. n.*], 2023. *TestDriven.io*. Accessed: 2023-04-10. Disponível em: https://testdriven.io/blog/python-openai-chatgpt/.

WARTMAN, S. A.; COMBS, C. D. Medical Education Must Move From the Information Age to the Age of Artificial Intelligence. **Academic Medicine**, v. 93, n. 8, p. 1107–1109, ago. 2018. DOI: 10.1097/ACM.0000000000002044.

WOLLNY, Sebastian *et al.* Are We There Yet? - A Systematic Literature Review on Chatbots in Education. **Frontiers in Artificial Intelligence**, v. 4, 2021. ISSN 2624-8212. DOI: 10.3389/frai.2021.654924. Disponível em: https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2021.654924.

ZHANG, Qing; CHEUNG, Elizabeth S. T.; CHEUNG, Christian S. T. The Impact of Flipped Classroom on College Students' Academic Performance: A Meta-Analysis Based on 20 Experimental Studies. **Science Insights Education Frontiers**, v. 8, n. 2, p. 1059–1080, abr. 2021. DOI: 10.2139/ssrn.3838807. Disponível em: https://ssrn.com/abstract=3838807.